

AI-gestuurd dependency dashboard voor automatische analyse en prioritering van software-updates.

Jelle Vandriessche.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Aelbrecht

Co-promotor: Dhr. B. Pattyn

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Deze bachelorproef ontwikkelt een aparte webapplicatie (dashboard) met een from-scratch ontworpen AI-agent om dependency-updates automatisch te detecteren, analyseren en prioriteren voor het kernteam dat platform- en dependencybeheer opneemt binnen een bedrijf. Vanuit de vastgestelde problematiek — veel manueel werk bij het interpreteren van changelogs, ophopende technische schuld en beperkte context bij updates — controleert de applicatie GitHub-repositories op gebruikte dependencies, haalt versie-informatie op uit publieke registries (zoals npm) en laat de AI-agent changelogs en release notes samenvatten met indicatie van criticiteit (security, bugfix, feature) en risico-impact.

Via een systematische aanpak met requirements-analyse, architectuurontwerp, proof-of-concept implementatie en gerichte vergelijking van AI-aanpakken wordt onderzocht hoe deze oplossing de manuele beoordelingslast kan verminderen en het overzicht op technische schuld kan verbeteren. De evaluatie met het kernteam focust op bruikbaarheid van het dashboard, duidelijkheid van AI-samenvattingen en ondersteuning bij prioritering. Verwacht wordt dat de aanpak een efficiënter, transparanter dependencybeheerproces oplevert dat herhaalbaar is en kan worden opgeschaald naar andere teams binnen de organisatie.

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	3
1.4 Opzet van deze bachelorproef	4
2 Stand van zaken	6
2.1 Dependency-updates als onderhoudslast	6
2.1.1 Kwetsbare en verouderde dependencies in de praktijk	6
2.1.2 Technische schuld als gevolg van uitgesteld onderhoud	7
2.1.3 De kloof tussen risico en actie	7
2.2 Bestaande oplossingen en hun grenzen	8
2.2.1 Dependencybots: wat ze wel en niet doen	8
2.2.2 De ontbrekende schakel: context en beslissingsondersteuning	8
2.2.3 Bijzondere uitdagingen bij private repositories en registries	9
2.3 AI-agents en workflow automatisering	9
2.3.1 Wat maakt een taak geschikt voor een AI-agent	9
2.3.2 Agentic design: tools, orkestratie en feedbackloops	10
2.3.3 Code gedreven versus low-code orkestratie	10
2.3.4 Tavily als zoek- en extractielaag	10
2.4 Platformen en ontwerpkeuzes	11
2.4.1 Overzicht van platformcategorieën	12
2.4.2 Code-gedreven frameworks	12
2.4.3 Low-code workflowtools	13
2.4.4 API-gestuurde agentconfiguratie	13
2.5 Synthese: de onderzoeksniche	13
3 Methodologie	15
3.1 Fase 1 - Verdiepende literatuurstudie en probleemverkenning	15
3.2 Fase 2 - Requirements-analyse en architectuurontwerp	16
3.3 Fase 3 - Implementatie van het proof-of-concept	16

3.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak .	17
3.5	Fase 5 - Evaluatie en rapportering	18
4	Requirementsanalyse	20
4.1	Stakeholders	20
4.2	Functionele vereisten	21
4.2.1	Authenticatie en gebruikersbeheer	21
4.2.2	Projectbeheer	21
4.2.3	Dependency scanning en versiebeheer	22
4.2.4	AI-analyse en samenvatting	22
4.2.5	Dashboard en visualisatie	23
4.2.6	Notificaties en communicatie	23
4.2.7	Regels en beslissingsondersteuning	24
4.3	Niet-functionele vereisten	24
4.3.1	Beveiliging	24
4.3.2	Prestaties	24
4.3.3	Schaalbaarheid	25
4.3.4	Betrouwbaarheid en foutafhandeling	25
4.3.5	Onderhoudbaarheid	25
4.3.6	Bruikbaarheid	25
4.3.7	Traceerbaarheid en audit	26
4.3.8	Portabiliteit en installatie	26
4.4	Samenvattende MoSCoW-prioritering	26
5	Ontwerp en implementatie van het AI-gestuurde dashboard	27
5.1	Architectuur van het systeem	27
5.2	Implementatie van de Mastra-agent	30
5.3	Implementatie van de n8n-workflow	33
5.4	Implementatie van de OpenAI-agent-builder	36
5.5	Integratie met de webapplicatie	39
6	Resultaten en evaluatie	41
6.1	Evaluatiekader	41
6.2	Globale prestaties	42
6.3	Per agent	42
6.4	Vergelijking per testcase	43
6.5	Interpretatie van de resultaten	43
6.6	Antwoord op de onderzoeksvraag	44
7	Conclusie	45
A	Onderzoeksvoorstel	48
A.1	Inleiding	48
A.1.1	Probleemstelling	48

A.1.2	Onderzoeksvraag	49
A.1.3	Onderzoeksdoelstelling	50
A.2	Literatuurstudie	51
A.2.1	Dependency management en kwetsbare dependencies	51
A.2.2	Technische schuld en onderhoudslast	52
A.2.3	Dependencybots en hun beperkingen	52
A.2.4	AI, agents en workflow automatisering	52
A.2.5	Onderzoeksniche: AI-ondersteund dependencybeheer	53
A.3	Methodologie	54
A.3.1	Fase 1 - Verdiepende literatuurstudie en probleemverkenning.	54
A.3.2	Fase 2 - Requirements-analyse en architectuurontwerp	54
A.3.3	Fase 3 - Implementatie van het proof-of-concept	55
A.3.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak.	55
A.3.5	Fase 5 - Evaluatie en rapportering	56
A.4	Verwacht resultaat en conclusie	57

Bibliografie**59**

Lijst van figuren

3.1 Fasering en tijdsplanning van het onderzoek	19
5.1 Architectuuroverzicht DepGuard AI	28
5.2 Conceptueel datamodel DepGuard AI	29
5.3 AI-workflow in n8n	34
A.1 Fasering en tijdsplanning van het onderzoek	58

Lijst van tabellen

2.1	Overzicht van platformcategorieën voor AI-agenten	12
6.1	Overzicht van de gebruikte testcasegroepen	42
6.2	Samenvatting van de benchmarkresultaten per AI-variant	42

Lijst van codefragmenten

5.1	Configuratie van de dependencyAgent met de nodige tools voor de gekozen research strategie	31
5.2	Codefragment: fetchNpmInfoTool met npm-metadata en fallbacklogica	32
5.3	Codefragment: webChangelogSearchTool voor webzoekopdrachten naar changelogs	33
5.4	Codefragment: Next.js-route die de n8n-workflow aanroept	36
5.5	OpenAI dependencyAgent met system prompt en tool-loop	37
5.6	fetchNpmInfoTool: npm-metadata en GitHub-releases met CHANGELOG-fallback	38
5.7	webChangelogSearchTool: webzoekopdrachten naar changelogs . . .	38

1

Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden Pashchenko et al. (2018). In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld (Behutiye et al., 2024; Pashchenko et al., 2018). Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt Pashchenko et al. (2018). stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als *technische schuld* (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn Behutiye et al. (2024) en Ruiz et al. (2024). Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit Besker (2020). Om deze last te

verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn, Erlenhov et al. (2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd Erlenhov et al. (2022). Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van Springbok. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol.

1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform- en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt. <https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt. <https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem. <https://www.npmjs.com>

2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van Springbok?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen Springbok?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van Springbok?
7. In welke mate presteren de geselecteerde AI-agents effectief bij het analyseren en prioriteren van dependency-updates, gemeten in termen van nauwkeurigheid, bruikbaarheid van de adviezen en vermindering van manueel werk voor het kernteam?

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van Springbok voor geautomatiseerd dependencybeheer. De webapplicatie fungeert daarbij als losstaand platform naast GitHub en gebruikt GitHub voornamelijk als bron voor project- en dependency-informatie en als kanaal om eventuele vervolgacties (zoals pull requests of issues) te initiëren.

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registries (zoals npm);

- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van Springbok. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van Springbok;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen Springbok.

1.4. Opzet van deze bachelorproef

De bachelorproef volgt een klassieke onderzoeksstructuur en is als volgt opgebouwd:

- **Hoofdstuk 1** Dit hoofdstuk beschrijft de context, de probleemstelling, de onderzoeksvragen en de doelstellingen van de bachelorproef.
- **Hoofdstuk 2** In dit hoofdstuk wordt de relevante literatuur rond dependency management, bestaande tools en oplossingen, technische schuld en AI-agentframeworks en workflow-automatisering geanalyseerd.
- **Hoofdstuk 3** Dit hoofdstuk bespreekt de onderzoeksaanpak, het ontwerp van de systeemarchitectuur, de selectie van platformen en de implementatie van de proof-of-concept.
- **Hoofdstuk 4** Dit hoofdstuk bevat de requirementsanalyse voor het AI-gestuurde dependencydashboard in de context van Springbok.
- **Hoofdstuk 5** Dit hoofdstuk licht het ontwerp en de implementatie toe van het proof-of-concept, inclusief de Mastra-agent en de n8n-workflow.
- **Hoofdstuk 6** In dit hoofdstuk worden de resultaten van de implementatie en experimenten gepresenteerd, en wordt de oplossing geëvalueerd op basis van de gedefinieerde vereisten en criteria.

- **Hoofdstuk 7** Dit hoofdstuk vat de belangrijkste bevindingen samen en formuleert aanbevelingen voor verdere uitwerking en mogelijke toekomstige verbeteringen.

2

Stand van zaken

Dit hoofdstuk schetst de stand van zaken rond dependencybeheer, technische schuld, bestaande automatiseringsoplossingen en recente ontwikkelingen in AI-agents en workflow-automatisering. Het doel is om stap voor stap te onderbouwen waarom dependency-updates vandaag een relevant onderhoudsprobleem vormen, waarom klassieke oplossingen slechts gedeeltelijk verlichting bieden en waarom er nog ruimte is voor een AI-ondersteunde aanpak. Op basis van die literatuur wordt uiteindelijk de onderzoeksniche van deze bachelorproef afgebakend.

2.1. Dependency-updates als onderhoudslast

Softwareprojecten steunen in toenemende mate op open-source dependencies, waardoor het onderhoud van externe bibliotheken een structureel onderdeel is geworden van moderne softwareontwikkeling. Die afhankelijkheid verhoogt de snelheid waarmee teams kunnen bouwen, maar maakt projecten tegelijk kwetsbaar voor verouderde of onveilige versies van componenten die buiten de eigen codebasis liggen (Pashchenko et al., 2018). Dependencybeheer is daardoor niet langer een louter technische beheertaak, maar een voortdurende onderhoudsverantwoordelijkheid die rechtstreeks samenhangt met stabiliteit, veiligheid en evoleerbaarheid van software.

2.1.1. Kwetsbare en verouderde dependencies in de praktijk

Onderzoek toont aan dat kwetsbaarheden in externe libraries niet uitzonderlijk zijn en dat veel van die problemen in principe relatief eenvoudig op te lossen zijn door te upgraden naar een veiligere versie Pashchenko et al. (2018). Toch voeren ontwikkelteams dergelijke updates in de praktijk vaak niet consequent door. Dat wijst erop dat het probleem niet alleen in de aanwezigheid van kwetsbare dependencies zit, maar ook in de manier waarop teams de bijhorende informatie verwerken,

beoordelen en opvolgen. De uitdaging ligt dus niet enkel in detectie, maar vooral in het omzetten van update-informatie naar tijdige actie.

Daarnaast maakt het onderscheid tussen daadwerkelijk in productie gebruikte dependencies en packages die enkel in ontwikkeling of testing voorkomen duidelijk dat niet elke kwetsbaarheid dezelfde impact heeft Pashchenko et al. (2018). Juist daarom is context belangrijk: teams moeten kunnen inschatten welke dependency werkelijk risico oplevert en welke update effectief prioriteit verdient. Zonder dat onderscheid ontstaat al snel een overload aan signalen die moeilijk te vertalen is naar concrete onderhoudsbeslissingen.

2.1.2. Technische schuld als gevolg van uitgesteld onderhoud

Wanneer updates te lang worden uitgesteld, groeit de onderhoudslast uit tot technische schuld. Die schuld ontstaat wanneer korte termijnkeuzes, zoals het uitstellen van een update om tijd te besparen, op langere termijn leiden tot meer complexiteit, hogere kosten en grotere kans op fouten Behutiye et al. (2024) en Ruiz et al. (2024). In die zin is het uitblijven van dependency updates niet alleen een operationeel probleem, maar ook een bron van structurele accumulatie van onderhoudsrisico.

Empirisch werk toont aan dat technische schuld in industriële context concreet voelbaar wordt via verminderde productiviteit, vertragingen en een grotere foutkans bij softwarelevering Besker (2020). Voor dependencybeheer betekent dit dat elke gemiste update niet noodzakelijk meteen een incident veroorzaakt, maar wel bijdraagt aan een groeiende achterstand die latere wijzigingen moeilijker en duurder maakt. Het probleem is dus cumulatief: hoe langer teams wachten, hoe groter de inspanning wordt om de achterstand nog gecontroleerd in te halen.

2.1.3. De kloof tussen risico en actie

De kern van het probleem ligt niet in het ontbreken van updates, maar in de kloof tussen het signaleren van een risico en het effectief ondernemen van actie. Ontwikkelteams krijgen vaak wel informatie over nieuwe versies, securityfixes of breaking changes, maar die informatie is verspreid, technisch en tijdsintensief om te interpreteren. Daardoor wordt een update pas opgevolgd wanneer de noodzaak duidelijk zichtbaar wordt, terwijl het moment waarop een update het goedkoopst en veiligst is, vaak al voorbij is.

Die kloof verklaart waarom dependency updates vaak blijven liggen, zelfs wanneer de nood aan onderhoud rationeel duidelijk is. Teams moeten niet alleen weten dat er een update bestaat, maar ook wat de inhoud en impact van die update is. Precies daar ontstaat de link met de rest van dit hoofdstuk: bestaande oplossingen proberen die informatie deels te structureren, maar ze lossen het interpretatieprobleem niet volledig op.

2.2. Bestaande oplossingen en hun grenzen

Momenteel bestaan er reeds meerdere vormen van ondersteuning voor dependencybeheer, gaande van bots die updates detecteren tot tools die automatisch een pull request aanmaken. Die oplossingen verlagen de drempel om updates zichtbaar te maken, maar nemen het inhoudelijke beoordelingswerk nog niet weg. Daardoor verschuift de taak van detectie wel naar automatische systemen, maar blijft de interpretatie van changelogs, compatibiliteit en risico's grotendeels bij de ontwikkelaar liggen Erlenhov et al. (2022) en Pashchenko et al. (2018).

2.2.1. Dependencybots: wat ze wel en niet doen

Dependencybots zoals Dependabot zijn nuttig omdat ze repetitieve en goed afgebakende taken automatiseren. Ze signaleren nieuwe versies, openen pull requests en zorgen ervoor dat updates niet volledig over het hoofd worden gezien. In die zin vormen ze een belangrijke stap in het verlagen van de operationele last rond afhankelijkheden, vooral omdat ze een groot deel van het handmatige opvolgwerk uit handen nemen.

Tegelijk blijven deze bots beperkt in hun inhoudelijke bijdrage. Ze kunnen doorgaans aantonen dat er een update is, maar niet overtuigend uitleggen dat de implicaties ervan zijn. Voor ontwikkelaars blijft het dus nodig om changelogs, release notes en migratie-informatie zelf door te nemen om te bepalen of een update veilig, dringend of complex is. De literatuur suggereert daarmee dat automation op zichzelf niet volstaat wanneer de resterende taak vooral bestaat uit interpretatie en besluitvorming Erlenhov et al. (2022).

2.2.2. De ontbrekende schakel: context en beslissingsondersteuning

De echte meerwaarde lijkt te liggen in context: een systeem dat niet enkel meldt dat er iets veranderd is, maar ook helpt begrijpen wat die verandering betekent. Voor dependencybeheer is dat belangrijk omdat release notes vaak uit tekst, technische verwijzingen en impliciete aanames bestaan. Een ontwikkelaar moet die informatie vertalen naar de concrete context van het project, terwijl veel tools vandaag vooral focussen op de aanwezigheid van een update en niet op de inhoudelijke duiding ervan.

Daarmee ontstaat een duidelijk tekortkoming tussen detectie en beslissing. Bestaande oplossingen tonen aan dat updates bestaan en soms zelfs hoe dringend ze zijn, maar ze bieden zelden een compacte, contextbewuste samenvatting die het team helpt om snel te beslissen. Juist die vertaalslag van ruwe updates-informatie naar bruikbare beslissingsinformatie vormt een relevante onderzoeksvraag binnen dit domein.

2.2.3. Bijzondere uitdagingen bij private repositories en registries

De problematiek wordt complexer wanneer dependencies niet publiek beschikbaar zijn, maar via private repositories of interne repositories worden beheerd. In zulke omgevingen is automatische opvolging vaak lastiger, omdat tooling niet altijd dezelfde toegang en metadata krijgt als bij publieke packages. Daardoor kan update informatie minder volledig zijn of moeten teams alternatieve toegangsmechanismen voorzien om de nodige data überhaupt op te halen.

Dat maakt private ecosystemen extra relevant voor onderzoek naar dependencybeheer. Niet alleen is de hoeveelheid beschikbare informatie daar vaak beperkter, ook de operationele drempel om die informatie veilig te ontsluiten is hoger. Hierdoor worden de beperkingen van klassieke bots nog scherper zichtbaar: waar publieke packages vaak nog relatief goed ondersteund zijn, ontstaan in private omgevingen sneller blinde vlekken en manuele tussenstappen.

2.3. AI-agents en workflow automatisering

Wanneer de informatie over updates te verspreid of te contextafhankelijk wordt voor eenvoudige automatisering, komen AI-agents en workflow automatisering in beeld. Die technologieën zijn vooral interessant wanneer een taak bestaat uit meerdere stappen, waarbij informatie uit verschillende bronnen moet worden gecombineerd, geïnterpreteerd en teruggekoppeld naar een gebruiker of systeem Joshi (2025), Kim et al. (2024), en Zhu et al. (2025). In dat opzicht sluiten dependency updates goed aan bij het soort probleem waarvoor agentgebaseerde systemen ontworpen worden.

2.3.1. Wat maakt een taak geschikt voor een AI-agent

AI-agents zijn vooral nuttig in situaties waar een systeem die geen vaste handeling moet uitvoeren, maar iteratief informatie moet verzamelen, analyseren en samenvatten. Uit het onderzoek van Joshi (2025) en Zhu et al. (2025) blijkt dat agents bijzonder geschikt zijn voor taken met gestructureerde input, tekstuele context en meerdere beslismomenten. Dat maakt ze relevant voor softwarecontexten waarin logs, release notes, notificaties of documentatie samen moeten worden geïnterpreteerd.

Dependency analyse past goed in dat profiel. Een update bestaat vaak niet uit een duidelijk bericht, maar uit een combinatie van registry metadata, release notes, changelogs en soms aanvullende webdocumentatie. De taak is dus niet alleen informatief, maar ook selectief: niet elk detail is even relevant, en de bruikbare conclusie hangt af van de combinatie van bronnen. Daardoor lijkt dit domein geschikt voor een agent die informatie eerst verzamelt en vervolgens compact teruggeeft.

2.3.2. Agentic design: tools, orkestratie en feedbackloops

In de literatuur over agentarchitecturen keren telkens dezelfde bouwstenen terug: een taalmodel, tools, een vorm van orkestratie en een feedbacklus waarin eerdere resultaten worden meegenomen in volgende stappen Kim et al. (2024) en Zhu et al. (2025). Tools laten een agent toe om externe bronnen op te vragen of acties uit te voeren, terwijl orkestratie bepaalt in welke volgorde die stappen gebeuren. Feedbackloops maken het mogelijk om nieuwe informatie opnieuw te integreren wanneer de eerste bron onvoldoende blijkt.

Voor het type probleem dat in deze bachelorproef centraal staat, is dat relevant omdat dependencyinformatie vaak niet volledig in een bron zit. Een agent moet daarom kunnen beginnen bij een primaire bron en indien nodig overschakelen naar aanvullende context. De literatuur ondersteunt zo het idee dat agentgedrag niet noodzakelijk draait om autonomie als doel op zich, maar om gecontroleerde iteratie in informatieverzameling en interpretatie.

2.3.3. Code gedreven versus low-code orkestratie

De manier waarop zulke agentische processen worden georganiseerd, verschilt sterk tussen code gedreven en low-code benaderingen. Code gedreven systemen bieden doorgaans meer controle over de volgorde van stappen, de dataflow en de fouthandeling, terwijl low-code platformen nadruk leggen op visuele samenstelling, sneller configuratie en toegankelijkheid voor een breder profiel van gebruikers Wali et al. (2025). Beide benaderingen kunnen nuttig zijn, maar ze leggen andere accenten in termen van flexibiliteit, transparantie en onderhoudbaarheid.

Voor dependencybeheer is dat onderscheid relevant omdat de taak enerzijds voldoende gestructureerd is om in een workflow te vatten, maar anderzijds genoeg nuance bevat om baat te hebben bij expliciete controle over toolgebruik en bronselectie. De literatuur suggereert dus niet een enige juiste aanpak, maar wel dat de keuze tussen een code gedreven of low-code orkestratie mee bepaald wordt door de gewenste balans tussen controle en snelheid.

2.3.4. Tavily als zoek- en extractielaag

Naaste workflowtools en agentframeworks bestaan ook bouwstenen die specifiek gericht zijn op het ophalen en voorbereiden van externe webinformatie voor AI-toepassingen. Tavily is een zoekdienst die ontworpen is voor AI-agents en RAG-workflows en biedt onder meer een web search API, een extractie-API en mogelijkheden voor site crawling en domeinfiltering Tavily (2026c, 2026d). In deze bachelorproef is Tavily relevant omdat de tool niet enkel zoekresultaten teruggeeft, maar deze ook structureert op een manier die beter aansluit bij gebruik door taalmodellen, bijvoorbeeld via titels, URL's, samenvattingen en scoringsinformatie Tavily (2026d) en Tavily AI (2025).

De keuze voor Tavily wordt vooral ingegeven door vier criteria. Ten eerste sluit de

tool goed aan bij het beoogde gebruik in een AI-gestuurde analyseketen, omdat ze expliciet ontworpen is voor agentische zoekscenario's en daardoor beter past dan een algemene zoekmachine. Ten tweede ondersteunt Tavily gerichte filtering op domein en inhoud, wat belangrijk is om officiële documentatie, release notes en changelogs te prioriteren boven minder betrouwbare bronnen Tavily (2026b, 2026d). Ten derde levert de tool output die rechtstreeks bruikbaar is voor verdere tekstverwerking, waardoor minder nabewerking nodig is dan bij klassieke zoekresultaten. Ten vierde past de tool binnen een workflow waarin actuele informatie snel moet worden opgehaald zonder dat de agent zelf uitgebreide scraping- of extractielogica moet bevatten.

Voor toepassingen zoals dependency-analyse is dat relevant omdat release notes, migratiegidsen en changelogs niet altijd rechtstreeks beschikbaar zijn via gestructureerde registries. In zulke gevallen kan een zoeklaag zoals Tavily helpen om recente en contextuele informatie op te halen uit officiële documentatie of andere relevante webbronnen. De tool ondersteunt bovendien een meer gecontroleerde bronselectie dan een generieke zoekmachine, waardoor de kans kleiner wordt dat irrelevante of verouderde informatie in de analyse terechtkomt Tavily (2026b, 2026d). Daardoor vormt Tavily een nuttige schakel wanneer een agent of workflow verder moet zoeken dan de primaire package-metadata of repository-informatie.

Binnen een geautomatiseerde analyseketen kan Tavily dus functioneren als aanvullende informatiebron tussen de ruwe dependencygegevens en de uiteindelijke AI-samenvatting. In vergelijking met alternatieven zoals een klassieke zoekmachine of zelf opgebouwde scrapinglogica biedt Tavily in deze context vooral meer doelgerichtheid, betere structuur van de output en minder implementatiecomplexiteit. De literatuur en documentatie suggereren daarom dat dit type zoekdienst vooral waardevol is wanneer het systeem actuele, brongebonden informatie nodig heeft die nog niet in de primaire databron aanwezig is Tavily (2026a, 2026c). In die zin vult Tavily de kloof tussen eenvoudige metadata-opvraging en diepere tekstuele interpretatie van online bronnen.

2.4. Platformen en ontwerpkeuzes

Binnen de literatuur rond AI-agents en workflowautomatisering komen verschillende platformcategorieën naar voren die elk een andere balans bieden tussen controle, ontwikkelsnelheid en integratiegemak. Voor deze bachelorproef zijn drie categorieën relevant: code-gedreven agentframeworks, low-code workflowtools en API-gestuurde agentconfiguraties. Binnen die categorieën worden onder meer Mastra AI, n8n en de agent- en toolgebaseerde configuratiemogelijkheden van grote taalmodelplatformen als representatieve opties besproken AI, 2025, 2026; Hatchworks, 2025; n8n GmbH, 2025; OpenAI, 2026a, 2026b, 2026c; Proser, 2025; The AI Builders, 2025.

De keuze om deze platformcategorieën eerst breed te schetsen is belangrijk om-

dat zij elk een andere ontwerpfilosofie vertegenwoordigen. Code-gedreven frameworks bieden maximale controle en uitbreidbaarheid, workflowtools focussen op visuele orkestratie en snelle integratie, en API-gestuurde configuraties laten toe om agentgedrag direct te sturen vanuit een model- en toolset. Door deze categorieën naast elkaar te plaatsen, wordt duidelijk welke afwegingen spelen bij de selectie van een geschikte aanpak voor dependencybeheer.

2.4.1. Overzicht van platformcategorieën

Tabel 2.1 geeft een beknopt overzicht van de voornaamste platformcategorieën, hun kenmerken en representatieve voorbeelden. Dit overzicht vormt de basis voor de latere vergelijking en onderbouwing van de uiteindelijke platformkeuze.

Tabel 2.1: Overzicht van platformcategorieën voor AI-agenten

Categorie	Kenmerken	Voorbeelden
Code-gedreven frameworks	Hoge controle over agentlogica, tools en integratie; geschikt voor uitbreidbare en technisch nauwkeurige implementaties.	Mastra AI
Low-code workflowtools	Visuele orkestratie van triggers, databronnen en werkingsstappen; sterk in snelle integratie en procesautomatisering.	n8n
API-gestuurde configuraties	Directe aansturing via model, instructies en tool/function calling; compact en flexibel voor gecontroleerde agentflows.	OpenAI Agent Builder, function calling, tool calling

2.4.2. Code-gedreven frameworks

Code-gedreven frameworks zoals Mastra AI zijn interessant wanneer de ontwikkelaar maximale controle wil over de agentlogica, de tools en de integratie met een bestaande codebase. De literatuur rond agentframeworks benadrukt dat zulke systemen vooral waardevol zijn wanneer uitbreidbaarheid belangrijk is AI (2025, 2026) en The AI Builders (2025). Hun sterkte ligt in technische precisie en in de mogelijkheid om stappen, tools en antwoorden nauwkeurig te structureren. Dat maakt dit type platform aantrekkelijk voor taken waarbij de kwaliteit van de output niet alleen afhangt van het taalmodel, maar ook van de manier waarop in-

putbronnen worden gekozen en gecombineerd. Voor dependencybeheer is dat relevant omdat updates inhoudelijk moeten worden samengevat zonder essentiële details te verliezen. Code-gedreven benaderingen bieden in dat geval de mogelijkheid om de tussenstappen expliciet te modelleren.

2.4.3. Low-code workflowtools

Low-code workflowtools zoals n8n richten zich op het visueel verbinden van data-bronnen, triggers en verwerkingsstappen. De literatuur benadrukt dat zulke tools vooral goed passen bij repetitieve, eventgedreven processen waarin extreme systemen aan elkaar moeten worden gekoppeld Hatchworks (2025), n8n GmbH (2025), en Proser (2025). Hun waarde ligt minder in diepgaande programmatische controle en meer in snelle samenstelling en duidelijke visualisatie van de processtroom. Voor AI-ondersteunde dependencyanalyse betekent dit dat low-code tools goed bruikbaar zijn wanneer de nadruk ligt op integratie en automatisering van bestaande stappen. Ze maken het eenvoudiger om informatie uit verschillende bronnen samen te brengen en door te geven aan een model, maar bieden minder nuance dan een volledige code-gedreven aanpak wanneer de logica complexer wordt. De literatuur toont daarmee aan dat low-code vooral sterk is in orkestratie, maar niet noodzakelijk in nauwkeurige agentcontrole.

2.4.4. API-gestuurde agentconfiguratie

Een derde optie is een meer directe, API-gestuurde aanpak waarbij een taalmodel via function calling of tool calling zelf stappen kan initiëren binnen een gecontroleerde loop OpenAI (2026a, 2026b, 2026c). In die benadering wordt de agent niet noodzakelijk ingebed in een zwaar framework, maar geconfigureerd via een model, instructies en expliciet gedefinieerde tools. Dat levert een compacte maar krachtige manier op om iteratieve informatieverwerking te organiseren.

Deze aanpak is vooral interessant wanneer de ontwikkelaar de agentflow nauw wil afstemmen op de taak, maar geen extra orkestratielaag wil toevoegen die de controle overneemt. De literatuur wijst erop dat dergelijke API-gestuurde agentconfiguraties nuttig zijn voor taken waarin modellen meerdere tools achter elkaar moeten kunnen aanroepen en de resultaten daarvan in een gestructureerde output moeten samenbrengen OpenAI (2026b, 2026c). Daardoor vormen ze een relevante ontwerpoptie binnen het bredere landschap van AI-automatisering.

2.5. Synthese: de onderzoeksniche

De literatuur maakt duidelijk dat dependency updates een reële onderhoudslast vormen. Kwetsbare en verouderde dependencies verhogen het risico op technische schuld, terwijl het uitstellen van updates de onderhoudsachterstand verder doet groeien Behutiye et al. (2024), Besker (2020), Pashchenko et al. (2018), en Ruiz et al. (2024). Tegelijk tonen bestaande oplossingen aan dat detectie op zichzelf niet

voldoende is: dependencybots maken updates zichtbaar, maar laten de inhoudelijke interpretatie grotendeels aan ontwikkelaars over Erlenhov et al. (2022).

Daarmee ontstaat een duidelijke tekortkoming. Er is nood aan ondersteuning die niet enkel meldt dat er een update bestaat, maar ook helpt om de betekenis van die update snel te begrijpen binnen de context van een concreet project. AI agents en workflowautomatisering bieden hiervoor een interessant antwoord, omdat ze informatie uit meerdere bronnen kunnen combineren en samenvatten Joshi (2025), Kim et al. (2024), Wali et al. (2025), en Zhu et al. (2025). De literatuur wijst dus op een volgende stap tussen eenvoudige detectietools en volledig handmatige evaluatie. De combinatie die in deze bachelorproef onderzocht wordt, situeert zich precies in die ruimte: een AI ondersteunde laag bovenop dependencyopvolging die changelog- en update-informatie niet alleen verzamelt, maar ook bruikbaar tracht te maken voor beslissingsvorming. Daarmee wordt beoogd om de kloof tussen risicosignalering en actie te verkleinen. Daarmee wordt beoogd om de kloof tussen risicosignalering en actie te verkleinen. De relevante onderzoeksniche ligt dus niet in het louter detecteren van updates, maar in het ondersteunen van de interpretatie ervan zodat teams sneller en gericht kunnen beslissen over onderhoud.

3

Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

3.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van Springbok geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

3.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kritici-teit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

3.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een afzonderlijke webapplicatie waarin per project dependencies worden bijgehouden en verrijkt met informatie uit externe bronnen, zoals package registries en repositorygegevens. De applicatie wordt opgezet in een realistische context, zodat de proof-of-concept kan worden getest op basis van echte projectgegevens en representatieve update-informatie.

Parallel wordt een AI-ondersteunde analysekern geïntegreerd die changelogs, release notes en andere relevante update-informatie verwerkt en omzet naar compacte samenvattingen en indicaties van de aard van een update. In deze fase worden meerdere varianten van die AI-integratie uitgewerkt, zodat later een gerichte vergelijking kan worden gemaakt tussen verschillende vormen van agent- en workflow-orkestratie. De frontend visualiseert vervolgens de verzamelde dependencygegevens, de AI-gegenereerde toelichting en de voorgestelde prioritering. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar project- en dependencygegevens;
- meerdere geïmplementeerde varianten van de AI-ondersteunde analysekern;
- technische documentatie over de implementatie, architectuur en integratiepunten.

3.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requirements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Mastra AI onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeprobeerde varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

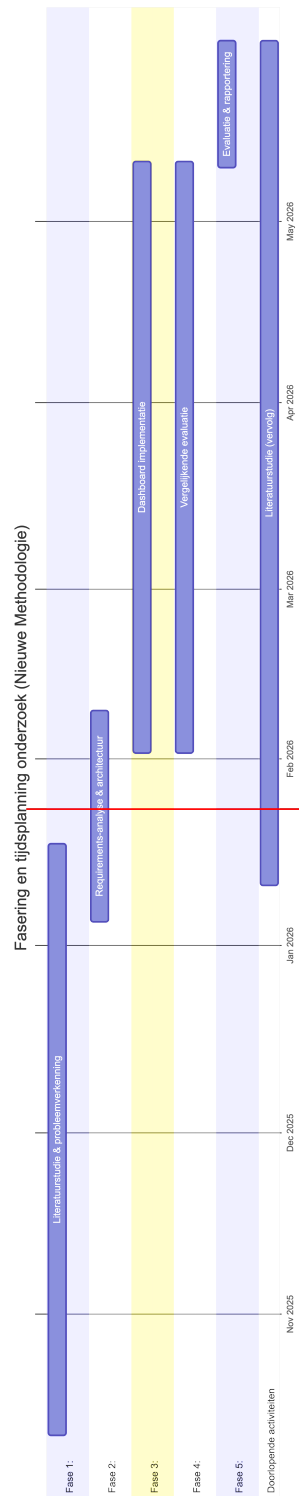
3.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd op basis van de bevindingen uit de implementatie en de vergelijking van de verschillende varianten. Daarbij wordt gekeken naar de bruikbaarheid van het dashboard, de kwaliteit van de AI-gegenereerde samenvattingen, de technische haalbaarheid van de oplossing en de mate waarin het systeem aansluit bij de vooropgestelde vereisten. Waar relevant wordt deze evaluatie aangevuld met feedback van de co-promotor of andere betrokkenen uit de bedrijfscontext.

De bevindingen uit deze evaluatie worden samengebracht om de onderzoeksvragen te beantwoorden en de ontwerpkeuzes te reflecteren. Deze fase mondt uit in de finale rapportering van de bachelorproef, waarin probleemstelling, methode, implementatie, resultaten en conclusies in samenhang worden gepresenteerd. Deliverables van fase 5 zijn:

- een synthese van de evaluatiebevindingen en aandachtspunten;
- de uitgewerkte bachelorproef met geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling of vervolgonderzoek.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur 3.1.



Figuur 3.1: Fasering en tijdsplanning van het onderzoek

4

Requirementsanalyse

Deze requirementsanalyse beschrijft de functionele en niet-functionele vereisten van DepGuard AI, de AI-gestuurde dependencydashboardoplossing die in deze bachelorproef wordt ontwikkeld. De analyse bouwt voort op de probleemstelling, literatuurstudie en het overleg met de co-promotor. Het platform wordt opgezet als een losstaande webapplicatie, onafhankelijk van bestaande tools zoals Dependabot, en richt zich op het samenvatten van release notes, het beoordelen van de criticiteit van updates en het ondersteunen van het kernteam bij het prioriteren van dependency updates.

4.1. Stakeholders

De belangrijkste stakeholders van DepGuard AI zijn:

- **Kernteam voor platform- en dependencybeheer:** primaire gebruikers, verantwoordelijk voor het opvolgen en doorvoeren van dependency-updates en dagelijks gebruik van het dashboard voor overzicht, prioritering en AI samenvattingen.
- **Developers binnen projecten:** secundaire gebruikers die aan individuele projecten werken, update-adviezen raadplegen en projectleden en dependencies beheren.
- **Co-promotor / bedrijfsbegeleider:** opdrachtgever vanuit Springbok, die de bruikbaarheid en kwaliteit van de oplossing evalueert en mee de requirements scherpstelt.
- **Promotor (HOGENT):** academische begeleider die de methodologie, uitvoering en kwaliteit van het eindwerk beoordeelt.

- **Externe platforms:** GitHub en de npm-registry leveren project- en versiemetadata via hun APIs en vormen zo cruciale bronnen voor het systeem.

4.2. Functionele vereisten

De functionele vereisten zijn gegroepeerd per domein en geprioriteerd volgens de MoSCoW-methode (Must Have, Should Have, Won't Have voor dit proof-of-concept).

4.2.1. Authenticatie en gebruikersbeheer

De volgende functionele vereisten gelden voor authenticatie en gebruikersbeheer.

- **F-01 (Must Have):** Gebruikersregistratie en login via e-mail en wachtwoord (Supabase Auth).
- **F-02 (Must Have):** Aanmaken van een bedrijfsaccount (multi-tenant).
- **F-03 (Must Have):** Uitnodigen van teamleden via e-mail met rolkoppeling (Resend API).
- **F-04 (Must Have):** Rollen per bedrijf: owner, project_lead, developer, tester (RBAC).
- **F-05 (Must Have):** Rollen per project: project_lead, developer (projectniveau RBAC).
- **F-06 (Should Have):** Uitnodiging accepteren via directe link zonder e-mail (fallbackflow).
- **F-07 (Could Have):** SSO / OAuth login (bijvoorbeeld GitHub of Google via Supabase OAuth).

4.2.2. Projectbeheer

De volgende functionele vereisten gelden voor projectbeheer.

- **F-08 (Must Have):** Aanmaken, bewerken en verwijderen van projecten (CRUD).
- **F-09 (Must Have):** Koppelen van een publieke GitHub-repository aan een project (GitHub-integratie).
- **F-10 (Must Have):** Uploaden van een `package.json` om dependencies in te laden.
- **F-11 (Must Have):** Toewijzen van teamleden aan projecten met projectrol.
- **F-12 (Must Have):** Overzicht van alle projecten per bedrijf met statusindicator.
- **F-13 (Could Have):** Archiveren of deactiveren van projecten.

4.2.3. Dependency scanning en versiebeheer

De volgende functionele vereisten gelden voor dependency scanning en versiebeheer.

- **F-14 (Must Have):** Automatisch detecteren van dependencies via GitHub (uit-lezen van `package.json` via de GitHub API).
- **F-15 (Must Have):** Ophalen van de meest recente versie via de npm-registry (`npmjs.org`).
- **F-16 (Must Have):** Vergelijken van huidige versie met meest recente versie (semver-vergelijking).
- **F-17 (Must Have):** Classificeren van updates als major, minor of patch (semver-classificatie).
- **F-18 (Must Have):** Opslaan van dependency-informatie (naam, versies, status) in Supabase Postgres.
- **F-19 (Should Have):** Manueel hertriggeren van een scan door de gebruiker.
- **F-20 (Could Have):** Automatisch periodiek scannen via een geplande job (cron).
- **F-21 (Won't Have):** Ondersteuning voor meerdere registries (bijv. PyPI, Maven).

4.2.4. AI-analyse en samenvatting

De volgende functionele vereisten gelden voor AI-analyse en samenvatting.

- **F-22 (Must Have):** AI-agent analyseert changelogs en release notes van npm en GitHub (Mastra-agent met LLM-backend).
- **F-23 (Must Have):** Genereren van een begrijpelijke samenvatting per dependency-update.
- **F-24 (Must Have):** Classificeren van het updatetype: security fix, bugfix, nieuwe feature, breaking change.
- **F-25 (Must Have):** Aanduiden of een update veilig, risicovol of kritiek lijkt (risico-indicatie).
- **F-26 (Must Have):** Opslaan van de AI-samenvatting per dependency in het veld `ai_summary`.
- **F-27 (Must Have):** Ondersteuning voor meerdere AI-providers (OpenAI, Google Gemini).

- **F-28 (Should Have):** Alternatieve AI-workflow via n8n (low-code orkestratie als vergelijking met de Mastra-agent).
- **F-28b (Should Have):** Alternatieve AI-agent via OpenAI Agent Builder, waarbij een gehoste agent met tools wordt geconfigureerd als tweede alternatief naast Mastra en n8n.
- **F-29 (Should Have):** Verrijken van AI-input met webcontext via Tavily.
- **F-30 (Should Have):** Hertriggere van AI-analyse op verzoek van de gebruiker.
- **F-31 (Could Have):** Gestructureerde output met migratiestappen en deprecation-warnings.

4.2.5. Dashboard en visualisatie

De volgende functionele vereisten gelden voor het dashboard en visualisatie.

- **F-32 (Must Have):** Overzichtsdashboard met statistieken (verouderde packages, security-issues, recente activiteit).
- **F-33 (Must Have):** Detailweergave per project met lijst van dependencies en hun status.
- **F-34 (Must Have):** Weergave van de AI-samenvatting per dependency in het dashboard.
- **F-35 (Must Have):** Visuele badge of kleurcodering voor major/minor/patch en risico-niveau.
- **F-36 (Should Have):** Filteren en sorteren van dependencies op status, updatetype en criticiteit.
- **F-37 (Could Have):** Exporteren van een dependency-rapport als PDF of CSV.

4.2.6. Notificaties en communicatie

De volgende functionele vereisten gelden voor notificaties en communicatie.

- **F-38 (Must Have):** E-mailnotificatie bij uitnodiging van een nieuw teamlid (Resend API).
- **F-39 (Should Have):** E-mailnotificatie bij detectie van een kritieke security-update.
- **F-40 (Could Have):** In-app notificatie of badge bij nieuwe updates per project.
- **F-41 (Won't Have):** Integratie met Slack of Microsoft Teams voor updateberichten.

4.2.7. Regels en beslissingsondersteuning

De volgende functionele vereisten gelden voor regels en beslissingsondersteuning.

- **F-42 (Must Have):** AI geeft een advies per dependency: “veilig te updaten”, “voorzichtigheid geboden” of “kritiek”.
- **F-43 (Should Have):** Mogelijkheid om een dependency handmatig als “genegeerd” of “gedaan” te markeren.
- **F-44 (Could Have):** Definieren van updatebeleid per project (bijvoorbeeld patch-updates automatisch goedkeuren).
- **F-45 (Won't Have):** Automatisch aanmaken van pull requests of GitHub-issues op basis van advies.

4.3. Niet-functionele vereisten

De niet-functionele vereisten zijn afgestemd op de gekozen technische stack (Next.js, Supabase, Mastra, n8n, OpenAI-agent builder en externe LLms) en de context van Springbok.

4.3.1. Beveiliging

- **NFR-01:** Row Level Security (RLS) in Supabase zorgt ervoor dat gebruikers uitsluitend data zien van het eigen bedrijf en de toegewezen projecten.
- **NFR-02:** Authenticatie is verplicht voor alle API-routes en dashboardpagina's via Next.js-middleware.
- **NFR-03:** API-sleutels (OpenAI, Google, Resend, GitHub) worden uitsluitend als omgevingsvariabelen opgeslagen en zijn nooit zichtbaar in de frontend.
- **NFR-04:** Uithodigingstokens zijn eenmalig geldig en vervallen na acceptatie.

4.3.2. Prestaties

- **NFR-05:** De end-to-end AI-analyse (van dependency-input tot opgeslagen samenvatting) wordt onder normale omstandigheden binnen ongeveer 30 seconden afgerond.
- **NFR-06:** Paginalaadtijden voor het dashboard blijven onder 2 seconden bij een dataset tot ongeveer 200 dependencies per bedrijf.
- **NFR-07:** De n8n-workflow en de Mastra-agent worden parallel geëvalueerd op doorlooptijd; de resultaten worden vastgelegd in de evaluatiefase.

4.3.3. Schaalbaarheid

- **NFR-08:** De multi-tenant architectuur laat toe om meerdere bedrijven, projecten en gebruikers onafhankelijk van elkaar te beheren.
- **NFR-09:** Opslag op Supabase (PostgreSQL) ondersteunt schaalbaarheid via managed infrastructuur en connection pooling.
- **NFR-10:** De AI-agentlaag is modulair en kan worden uitgebreid met extra tools, registries of LLM-providers zonder ingrijpende wijzigingen aan de webapplicatie.

4.3.4. Betrouwbaarheid en foutafhandeling

- **NFR-11:** Bij het ontbreken van release notes in GitHub wordt teruggevallen op bestanden zoals `CHANGELOG.md`, `CHANGES.md` of `HISTORY.md`.
- **NFR-12:** Als er geen changelog-informatie beschikbaar is, vermeldt de AI-agent dit expliciet in de samenvatting.
- **NFR-13:** API-fouten (GitHub, npm, LLM) worden gelogd en weergegeven als leesbare foutmeldingen in de UI.
- **NFR-14:** De Mastra-server en de Next.js-applicatie zijn onafhankelijk startbaar; bij uitval van Mastra blijft het dashboard bruikbaar zonder AI-analyse.

4.3.5. Onderhoudbaarheid

- **NFR-15:** De volledige codebase is getypeerd in TypeScript.
- **NFR-16:** Logica is opgesplitst per verantwoordelijkheid: AI-agentcode, API-routes en UI-componenten zijn in aparte modules gestructureerd.

4.3.6. Bruikbaarheid

- **NFR-17:** Het dashboard is responsive en bruikbaar op verschillende schermformaten.
- **NFR-18:** AI-samenvattingen zijn geschreven in begrijpelijke taal, gericht op developers zonder diepgaande AI-kennis.
- **NFR-19:** Statusbadges voor updatetype en risiconiveau zijn kleurgecodeerd en direct interpreteerbaar.
- **NFR-20:** Het onboardingproces (registratie, bedrijf aanmaken, project koppelen en eerste scan) kan in minder dan vijf minuten worden doorlopen.

4.3.7. Traceerbaarheid en audit

- **NFR-21:** Mastra voorziet tracing en logging van agentbeslissingen, tool-calls en modelinteracties, zodat het gedrag van de AI-agent kan worden geanalyseerd.
- **NFR-22:** Alle AI-samenvattingen worden met tijdstempel opgeslagen zodat de evolutie van adviezen per dependency navolgbaar is.
- **NFR-23:** Gebruikersacties zoals scannen, analyseren en uitnodigen zijn via Supabase auditeerbaar.

4.3.8. Portabiliteit en installatie

- **NFR-24:** De applicatie kan lokaal worden gestart via `npm run dev` (Next.js) en `npm run dev:mastra` (Mastra) na configuratie van de omgevingsvariabelen.
- **NFR-25:** De database-opzet is volledig scriptbaar en vereist geen manuele tabellaanmaak.
- **NFR-26:** De applicatie is inzetbaar op courante cloudplatformen, bijvoorbeeld Vercel voor Next.js en Supabase Cloud voor de databank.

4.4. Samenvattende MoSCoW-prioritering

In totaal worden 26 Must Have-, 9 Should Have-, 8 Could Have- en 4 Won't Have-vereisten onderscheiden. De Must Have-vereisten dekken de kernfunctionaliteit van DepGuard Ai (authenticatie, projectbeheer, scanning, AI-analyse, dashboard en basisadvies), terwijl de overige categorieën extra bruikbaarheid, gemak of toekomstige uitbreidingen beschrijven.

5

Ontwerp en implementatie van het AI-gestuurde dashboard

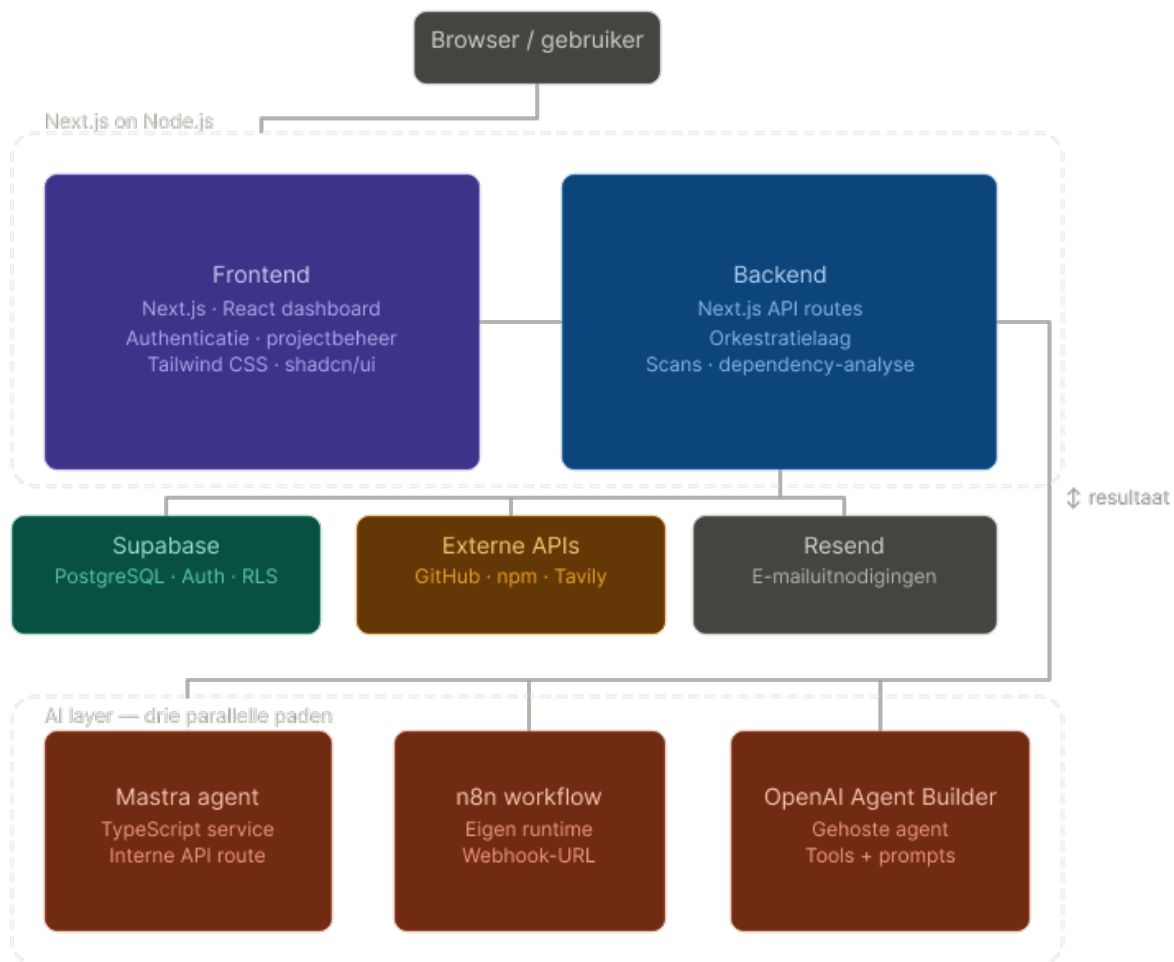
In dit hoofdstuk wordt beschreven hoe de concepten uit de literatuur en de requirements-analyse vertaald werden naar een werkende proof-of-concept. Eerst wordt de globale architectuur van het systeem toegelicht, gevolgd door de drie uitgewerkte AI-integraties en tot slot de koppeling met de webapplicatie. Op die manier vormt dit hoofdstuk de brug tussen het theoretische kader en de concrete realisatie van DepGuard AI.

5.1. Architectuur van het systeem

Het proof-of-concept voor DepGuard AI is opgezet als een moderne webarchitectuur met een duidelijke scheiding tussen frontend, backend, databank en AI-laag. De technologiekeuzes werden gemaakt op basis van de vereisten uit Fase 2 en de volgende criteria:

- **Frontend (Next.js, Tailwind CSS, shadcn/ui):** Next.js biedt een robuuste full-stack basis met server-side rendering en API-routes. Tailwind CSS en shadcn/ui zorgen voor consistente, toegankelijke componenten met minimale boilerplate.
- **Backend en databank (Supabase):** PostgreSQL met Row Level Security voldoet aan de eisen voor multi-tenant isolatie, realtime updates en eenvoudige integratie met Next.js.
- **AI-kern (Mastra, n8n, OpenAI Agent Builder):** Deze drie platformen vertegenwoordigen respectievelijk code-gedreven frameworks, low-code workflows en API-gestuurde configuraties. De selectie is gebaseerd op representativiteit, tool-ondersteuning, integratiegemak en documentatiekwaliteit.

Figuur 5.1 toont de globale systeemarchitectuur.



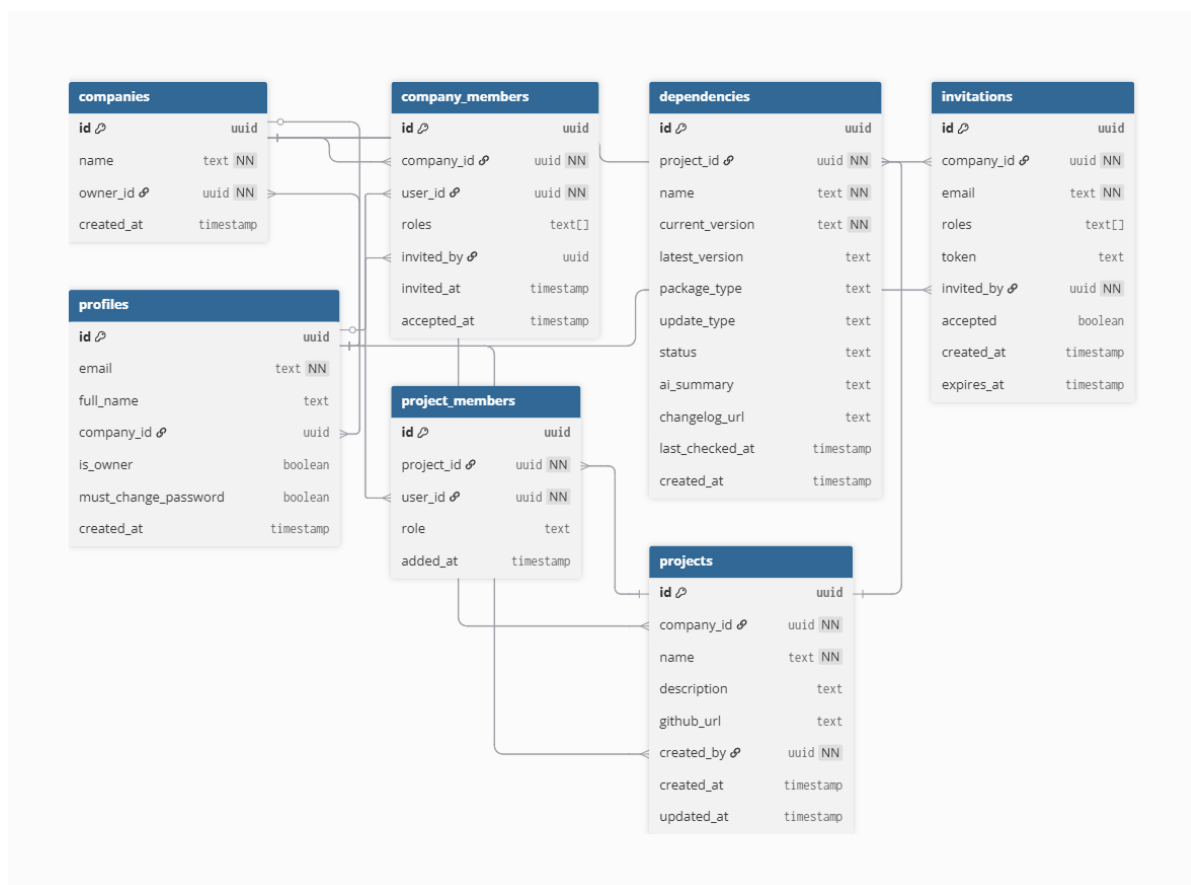
Figuur 5.1: Globale architectuur van DepGuard AI met frontend, backend, databank en AI-laag

De frontend is een Next.js dashboard dat authenticatie, projectbeheer en visualisatie van dependency- en AI-informatie verzorgt. Gebruikers kunnen bedrijven en projecten aanmaken, GitHub-repositories koppelen, dependency-scans opstarten en de door de AI gegenereerde samenvattingen en adviezen raadplegen. Voor de UI wordt gebruikgemaakt van Tailwind CSS en shadcn/ui, zodat componenten consistent en herbruikbaar zijn.

De backend bestaat uit een set Next.js API-routes die fungeren als orkestratielaag tussen de frontend, Supabase, externe APIs (GitHub, npm, Tavily) en de verschillende AI-componenten. Bij een scan leest de backend eerst de dependencyinformatie van een project (bijvoorbeeld uit `package.json` in GitHub), slaat de resultaten op in Supabase en roept vervolgens één van de AI-varianten aan: de mastra-agent, de n8n-workflow of de OpenAI-agent. De teruggestuurde samenvatting wordt gekoppeld aan de dependency en via het dashboard zichtbaar gemaakt voor de gebruiker.

Supabase verzorgt de persistente opslag van bedrijven, bedrijfleden, gebruikers, uitnodigingen, projecten, projectleden en dependencies, met Row Level Security om multi-tenant isolatie te garanderen. Het conceptuele datamodel is opgebouwd rond een centrale bedrijfsstructuur, waarvan gebruikers, projecten en dependenciesgegevens gekoppeld worden. De tabel `companies` bevat de kerngegevens van een organisatie en verwijst naar de eigenaar via `owner_id`. Via `company_members` worden bijkomende gebruikers aan een bedrijf gekoppeld, inclusief hun rol en het moment waarop ze werden uitgenodigd of geaccepteerd. De tabel `profiles` bewaart de profielgegevens van gebruikers en kan optioneel gekoppeld zijn aan een bedrijf.

De projectstructuur vertrekt vanuit `projects`, waar elk project behoort tot een bedrijf en een aanmaker heeft. Via `project_members` worden gebruikers aan projecten gekoppeld met een specifieke rol, zodat toegang en verantwoordelijkheden per project kunnen verschillen. De tabel `dependencies` bevat vervolgens de concrete dependencies per project, samen met versies, statusinformatie, AI samenvattingen en metadata zoals de changelog-URL en het tijdstip van de laatste controle. Ten slotte laat `invitations` toe om nieuwe gebruikers uit te nodigen voor een bedrijf via een tokengebaseerde uitnodiging. Figuur 5.2 toont hoe deze entiteiten samen het datamodel vormen waarop het dashboard steunt.



Figuur 5.2: Conceptueel ERD met bedrijven, projecten, leden en dependencies

De AI-laag bestaat uit drie parallelle paden die in aparte secties verder worden uitgewerkt. De Mastra-agent draait als TypeScript-service naast de Next.js-app en wordt aangeroepen via een interne API-route. De n8n-workflow draait in een eigen runtime en wordt aangesproken via een webhook-URL. De OpenAI-agent-builder wordt gebruikt om een gehoste agent te configureren met vergelijkbare tools en prompt. In alle gevallen verloopt de dataflow volgens hetzelfde patroon: dependencygegevens en eventueel changelog- en webcontext worden doorgestuurd naar de gekozen AI-component, de gestructureerde samenvatting komt terug naar de backend en wordt opgeslagen in Supabase. Hierdoor kan het dashboard de resultaten van de verschillende AI-aanpakken op een uniforme manier presenteren en vergelijken.

5.2. Implementatie van de Mastra-agent

Voor de eerste AI-integratie wordt gebruikgemaakt van Mastra, een TypeScript framework om AI-agents, tools en workflows te definiëren en te orchestreren. In de proof-of-concept wordt een `dependencyAgent` opgezet die specifiek is afgestemd op het analyseren van npm-dependency-updates. Deze agent combineert een taalmodel (bijvoorbeeld Google's `gemini-2.5-flash` of OpenAI's `gpt-4o-mini`) met twee eigen tools: `fetchNpmInfoTool` en `webChangelogSearchTool`.

De agent volgt een gestructureerde research-strategie: eerst haalt de `fetchNpmInfoTool` metadata en releasenotes op uit npm en GitHub; als die incompleet zijn, volgt de `webChangelogSearchTool` met een gerichte webzoekopdracht naar changelogs. Het resultaat wordt gecombineerd en volgens een strikt JSON-formaat geformatteerd, zodat ontwikkelaars direct actiegerichte inzichten krijgen over features, breaking changes, security fixes en migratiestappen. Codefragment 5.1 toont de definitie van de agent.

```
1 import { Agent } from "@mastra/core/agent";
2 import { google } from "@ai-sdk/google";
3
4 export const dependencyAgent = new Agent({
5   id: "DependencyAnalyzer",
6   name: "Dependency Analyzer",
7   instructions: `...`,
8   model: google("gemini-2.5-flash"),
9   tools: {
10     fetchNpmInfoTool,
11     webChangelogSearchTool,
12   },
13 });
```

Codefragment 5.1: Configuratie van de dependencyAgent met de nodige tools voor de gekozen research strategie

De `fetchNpmInfoTool` (zie Codefragment 5.2) is de primaire tool voor het ophalen van npm-metadata en GitHub-release notes. De tool zoekt eerst naar een exacte release-tag (bijvoorbeeld `v1.2.3`), valt terug op de laatste vijf releases als die niet bestaat, en probeert vervolgens CHANGELOG bestanden (`CHANGELOG.md`, `HISTORY.md`, ...) uit de repository te lezen. Dankzij fallback logica en robuuste foutafhandeling levert de tool altijd een gestructureerd resultaat, zelfs als er geen officiële release notes zijn.

```

export const fetchNpmInfoTool = createTool({
  id: "fetch-npm-info",
  inputSchema: z.object({
    packageName: z.string(),
    fromVersion: z.string(),
    toVersion: z.string(),
  }),
  execute: async ({ packageName, fromVersion, toVersion }) => {
    const npmData = await
      fetch(`https://registry.npmjs.org/${packageName}`).then(r =>
        r.json());
    const repoMatch = npmData.repository?.url?.match(/github\.com[/:](\[w.\,
      -\]+\[/\w.-\]+\)/);

    // exact tag, recente releases of CHANGELOG fallback
    if (repoMatch) {
      // ...
    }

    return {
      description: npmData.description || "",
      repository: npmData.repository?.url || "",
      releaseNotes: releaseNotes || "No release notes found on GitHub.",
      source,
    };
  },
});

```

Codefragment 5.2: Codefragment: fetchNpmInfoTool met npm-metadata en fallbacklogica

Als de npm-tool geen nuttige release notes vindt, volgt de `webChangeLogSearchTool` (Codefragment 5.3). Deze tool voert een gerichte webzoekopdracht uit naar changelogs via Tavily, Brave Search of DuckDuckGo, met domeinpriorisering voor officiële package-sites (bijvoorbeeld `next.js.org`, `angular.dev`). De tool prioriteert resultaten van officiële documentatie boven aggregators en levert een lijst van relevante titels, URL's en snippets.

```

export const webChangelogSearchTool = createTool({
  id: "web-changelog-search",
  inputSchema: z.object({
    packageName: z.string(),
    fromVersion: z.string(),
    toVersion: z.string(),
  }),
  execute: async ({ packageName, fromVersion, toVersion }) => {
    const query = buildQuery(packageName, fromVersion, toVersion);
    try {
      const provider = process.env.CHANGELOG_SEARCH_PROVIDER || "tavily"

      if (provider === "brave" && process.env.BRAVE_SEARCH_API_KEY) {
        return await searchWithBrave(query);
      }

      if (process.env.TAVILY_API_KEY) {
        return await searchWithTavily(query, packageName, toVersion);
      }

      return await searchWithDuckDuckGo(query);
    } catch (err) {
      return { results: [], query };
    }
  },
});

```

Codefragment 5.3: Codefragment: `webChangelogSearchTool` voor webzoekopdrachten naar changelogs

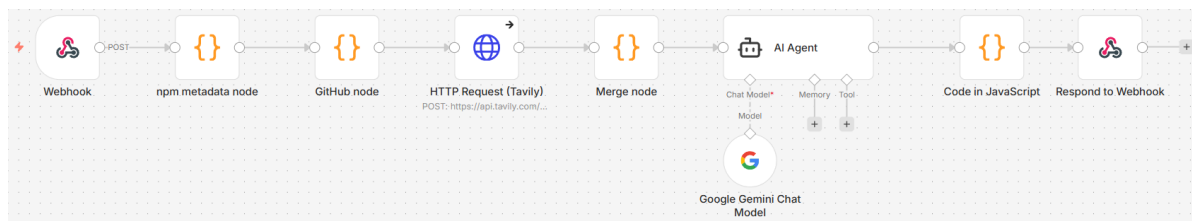
De Mastra-agent wordt vanuit de Next.js-backend aangeroepen via een API-route die de dependency-gegevens doorstuurt en de gestructureerde output opslaat in Supabase. Door de combinatie van deze twee tools en de expliciete instructies levert de agent betrouwbare, actionable samenvattingen die ontwikkelaars tijd besparen bij het beoordelen van updates.

5.3. Implementatie van de n8n-workflow

Als tweede AI-integratie wordt een n8n-workflow opgezet die dezelfde kernfunctie vervult als de Mastra-agent, maar dan in een low-code, visueel georkestreerde omgeving. De workflow ontvangt een HTTP-aanvraag vanuit de webapplicatie, verkrijgt de aangeleverde dependency-informatie met data uit npm, Github en een webzoekdienst (Tavily)¹, en laat vervolgens een AI-model (Gemini 2.5 flash) een samen-

¹Tavily is een API en infrastructuurlaag die AI-agents en LLM-toepassingen real-time toegang geeft tot webzoekopdrachten, webextractie en crawling.<https://www.tavily.com/>

vatting genereren van de relevante wijzigingen. Het resultaat wordt teruggestuurd naar de applicatie via de webhook-respons en opgeslagen in de databank. De workflow bestaat uit een keten van acht nodes (zie Figuur 5.3):



Figuur 5.3: Overzicht van de n8n-workflow die metadata, release notes, webcontext en AI-samenvatting combineert

- **Node 1: Webhook.** Deze node fungeert als ingangspunt voor de workflow en wordt getriggerd zodra de Next.js-backend een HTTP-aanvraag naar de n8n-URL stuurt. De node is geconfigureerd om te antwoorden via een aparte *Respond to Webhook*-node, zodat alle tussenliggende stappen eerst afgewerkt worden.
- **Node 2: npm-metadata (Code).** In deze code-node wordt de JSON-body van de inkomende request uitgelezen (`packageName`, `fromVersion`, `toVersion`) en wordt via `this.helpers.httpRequest` een call gedaan naar `https://registry.npmjs.org/{packageName}`. De node reconstrueert de repository-URL uit de npm-metadata en geeft een nieuwe JSON-object door met omschrijving, homepage en repository-informatie.
- **Node 3: Github-node (Code).** Deze node ontvangt de verrijkte npm-data, extraheert `owner/repo` uit de repository-URL en probeert via de Github API eerst release notes op te halen voor de specifieke tag (`v{toVersion}` of `toVersion`). Als die niet gevonden wordt, geeft de node de laatste releases op en concateneert de body-teksten tot een `releaseNotes`-veld. Als er ook geen releases zijn, wordt er nog geprobeerd een `CHANGELOG.md`, `CHANGES.md` of `HISTORY.md` rechtstreeks uit de repository te lezen. De node voegt `releaseNotes`, `changelogUrl` en `source` toe aan de JSON.
- **Node 4: HTTP Request (Tavily).** Via een HTTP-request-node wordt de Tavily API aangeroepen om webresultaten op te halen die extra context kunnen geven over de dependency of de update (bijvoorbeeld blogposts of documentatie rond de nieuwe versie).
- **Node 5: Merge-node (Code).** Deze node voegt de Github-data en Tavily-resultaten samen. De webresultaten worden omgezet naar een tekstveld `web-Snippets` (titel, URL en inhoud per resultaat), dat samen met de eerder verzamelde metadata en release notes wordt doorgegeven aan de AI-node.

- **Node 6: AI-agent (Gemini).** Een AI-node, geconfigureerd met het model `gemini-2.5-flash`, ontvangt `releaseNotes`, `webSnippets` en versie-informatie als input. Met behulp van een system prompt en user prompt genereert het model een gestructureerde samenvatting van de update (bijvoorbeeld features, breaking changes, security-aspecten en migratieadvies).
- **Node 7: Code-node (post-processing).** Deze node extraheert de relevante output uit de AI-respons (bijvoorbeeld `output` of `summary`) en zet die om naar een eenvoudige JSON-object met één veld `summary`, zodat de webhook-respons helder en compact blijft.
- **Node 8: Respond to Webhook.** De laatste node stuurt de `summary` terug als HTTP-respons naar de oorspronkelijke aanroeper. Aan de kant van de webapplicatie kan deze samenvatting vervolgens rechtstreeks worden opgeslagen in de databank.

De Next.js-backend roept de n8n-workflow aan via een dedicated route (vereenvoudigd weergegeven in Codefragment 5.4). Deze route valideert eerst de gebruiker met Supabase, leest de dependencygegevens uit de inkomende request (`dependencyId`, `name`, `currentVersion`, `latestVersion`) en stuurt die als JSON-payload door naar de n8n-webhook. De samenvatting die n8n terugstuurt wordt daarna opgeslagen in het veld `ai_summary` van de desbetreffende dependency.

```
1 export async function POST(request: Request) {
2   const supabase = await createClient();
3   const { data: { user } } = await supabase.auth.getUser();
4   if (!user) return NextResponse.json({ error: "Unauthorized" }, { status:
      401 });
5
6   const { dependencyId, name, currentVersion, latestVersion } = await
      request.json();
7
8   const n8nRes = await fetch(process.env.N8N_WEBHOOK_URL!, {
9     method: "POST",
10    headers: { "Content-Type": "application/json" },
11    body: JSON.stringify({ packageName: name, fromVersion: currentVersion,
      toVersion: latestVersion }),
12  });
13
14  const { summary } = await n8nRes.json();
15  await supabase.from("dependencies").update({ ai_summary: summary
      }).eq("id", dependencyId);
16
17  return NextResponse.json({ summary });
18 }
```

Codefragment 5.4: Codefragment: Next.js-route die de n8n-workflow aanroept

5.4. Implementatie van de OpenAI-agent-builder

Als derde AI-integratie wordt een eigen OpenAI-agent geïmplementeerd met de OpenAI API, die dependency updates analyseert via een expliciete tool loop. De agent gebruikt hetzelfde model als Mastra (gpt-4o-mini), gecombineerd met de tools `fetchNpmInfoTool` en `webChangelogSearchTool`, en volgt een gelijkaardige research-strategie: eerst npm/GitHub metadata, dan webzoekopdrachten bij onvolledige info. Codefragment 5.5 toont de kernconfiguratie met system prompt en agent-loop.

```
1 const SYSTEM_PROMPT = `
2 You are a dependency update analyst for software developers.
3 ## Research Strategy (follow IN ORDER)
4 ### Step 1 – fetch-npm-info
5 Always start by calling the fetch-npm-info tool...
6 ### Step 2 – web-changelog-search
7 Call when npm returns "No release notes found" ...
8 ## Output Format
9 ## [package] v[from] → v[to] ([major/minor/patch])
10 > Source: [URL]
11 ### New Features ### Breaking Changes
12 ### Security Fixes ### Deprecated
13 ### Migration Steps ### Safe to update?
14 `;
15
16 export async function analyzeDependency({name, currentVersion,
17   latestVersion}) {
18   const messages = [{role: "system", content: SYSTEM_PROMPT}, {role:
19     "user", ...}];
20   // Agentic loop - max 10 turns
21   for (let turn = 0; turn < 10; turn++) {
22     const response = await client.chat.completions.create({
23       model: "gpt-4o-mini", messages, tools, tool_choice: "auto"
24     });
25     // Handle tool calls: execute & feed back results
26     if (choice.finish_reason === "stop") return assistantMessage.content;
27   }
28 }
```

Codefragment 5.5: OpenAI dependencyAgent met system prompt en tool-loop

De `fetchNpmInfoTool` (Codefragment 5.6) haalt npm-metadata op, zoekt GitHub releases (exact tag of recentste) en valt terug op CHANGELOG parsing met fallback logica voor robuustheid.

```

export async function executeFetchNpmInfo(args) {
  const npmData = await
    fetch(`https://registry.npmjs.org/${args.packageName}`).then(r =>
      r.json());
  const repoMatch =
    npmData.repository?.url?.match(/github\.com[/:](?:[\w.-]+\w[\w.-]+)/);
  if (repoMatch) {
    // Exact tag, recent releases, CHANGELOG.md fallback
    for (const tag of [`v${args.toVersion}`, args.toVersion]) {
      const res = await fetch(`https://api.github.com/repos/${repoMatch[1]},
        /releases/tags/${tag}`);
      if (res.ok && data.body) { /* use release notes */ }
    }
  }
  return { description: npmData.description || "", releaseNotes:
    releaseNotes || "No release notes found" };
}

```

Codefragment 5.6: fetchNpmInfoTool: npm-metadata en GitHub-releases met CHANGELOG-fallback

Als npm onvoldoende releasenotes vindt, activeert de webChangelogSearchTool (Codefragment 5.7) een webzoekopdracht via Tavily, Brave of DuckDuckGo, met domeinprioriteit voor officiële sites zoals `nextjs.org` of `angular.dev`

```

export async function executeWebChangelogSearch(args) {
  const query = buildQuery(args.packageName, args.fromVersion,
    args.toVersion);
  try {
    const provider = process.env.CHANGELOG_SEARCH_PROVIDER || "tavily";

    if (provider === "brave" && process.env.BRAVE_SEARCH_API_KEY) {
      return await searchWithBrave(query);
    }
    if (process.env.TAVILY_API_KEY) {
      return await searchWithTavily(query, packageName);
    }
    return await searchWithDuckDuckGo(query);
  } catch {
    return { results: [], query };
  }
}

```

Codefragment 5.7: webChangelogSearchTool: webzoekopdrachten naar changelogs

De Next.js backend roept de OpenAI agent aan via een API route, analoog aan

Mastra en n8n: dependencygegevens worden doorgestuurd, de gestructureerde samenvatting opgeslagen in Supabase, en via het dashboard gepresenteerd. Deze custom implementatie biedt maximale controle over de toolflow en outputformaat, ideaal voor vergelijking met de framework-gebaseerde Mastra en n8n aanpakken.

5.5. Integratie met de webapplicatie

Deze sectie beschrijft hoe de verschillende onderdelen van DepGuard AI samenkomen in de webapplicatie en hoe dependencygegevens, AI-analyses en gebruikersinteracties in een uniforme gebruikerservaring worden geïntegreerd. De integratie is zodanig opgezet dat de frontend niet rechtstreeks communiceert met externe diensten of AI-componenten, maar steeds via de backendlaag werkt. Dat maakt het mogelijk om authenticatie, validatie, foutafhandeling en opslag centraal te beheren.

De webapplicatie vormt het centrale toegangspunt voor eindgebruikers. Vanuit het dashboard kunnen gebruikers projecten beheren, repositories koppelen, dependencyscans opstarten en de resultaten van AI-analyses bekijken. Daarbij fungeert de frontend in hoofdzaak als presentatielaag: gebruikersacties worden doorgestuurd naar de backendroutes, waarna de backend de nodige gegevens ophaalt, verwerkt en opnieuw in een gestandaardiseerd formaat aan de interface aanbiedt. Deze scheiding tussen gebruikersinterface en serverlogica verhoogt de onderhoudbaarheid en maakt het mogelijk om verschillende AI-implementaties op een uniforme manier in dezelfde applicatie te integreren.

Wanneer een gebruiker een project opent, haalt de applicatie eerst de bijhorende project- en dependencygegevens op uit Supabase. Per dependency worden onder meer de pakketnaam, huidige versie, laatst gekende versie, status en eventuele AI-samenvatting weergegeven. De frontend groepeerde deze informatie in een dashboardoverzicht, zodat gebruikers snel kunnen zien welke dependencies verouderd zijn en welke updates bijkomende aandacht vereisen. Indien voor een dependency al een analyse beschikbaar is, wordt die rechtstreeks vanuit de databank geladen en in de interface getoond, zonder dat de AI-component opnieuw moet worden aangeroepen.

Het opstarten van een nieuwe analyse verloopt op aanvraag vanuit de gebruikersinterface. Wanneer een gebruiker een dependency of project laat analyseren, verstuurt de frontend een verzoek naar een backendroute met de relevante parameters, zoals de pakketnaam en de huidige en nieuwste versie. De backend bepaalt vervolgens welke AI-variant beschikbaar is, verrijkt indien nodig de input met bijkomende metadata en stuurt de aanvraag door naar de juiste AI-component. Na verwerking wordt de teruggestuurde samenvatting opgeslagen in de tabel `dependencies`, zodat de analyse later opnieuw kan worden geraadpleegd zonder de volledige verwerkingsketen te herhalen.

De resultaten van de AI-analyse worden in de webapplicatie niet enkel als vrije tekst

weergegeven, maar ingebed in de bredere context van dependencybeheer. De samenvatting wordt gekoppeld aan de desbetreffende dependency en verschijnt samen met versie-informatie, updateclassificatie en eventuele bronverwijzingen, zoals een changelog-URL. Daardoor krijgt de gebruiker niet alleen een tekstuele uitleg van de wijziging, maar ook een concreet overzicht van welke dependency het betreft en welke bijkomende context beschikbaar is. Die koppeling is essentieel om AI-output bruikbaar te maken binnen een dashboardomgeving, aangezien de interpretatie pas waardevol wordt wanneer ze verbonden is met de operationele gegevens van het project.

Naast het ophalen en tonen van analyses ondersteunt de integratie ook de consistente vergelijking tussen de verschillende AI-aanpakken. Omdat alle varianten uiteindelijk een samenvatting in een gelijkaardig formaat teruggeven en die via dezelfde backendlaag in Supabase worden opgeslagen, kan het dashboard deze resultaten op een uniforme manier presenteren. Dat maakt het mogelijk om de verschillende implementaties binnen een applicatiecontext te evalueren, zonder voor elke aanpak een afzonderlijke gebruikersinterface te moeten uitwerken.

De integratie met de webapplicatie vervult daarmee een dubbele rol. Enerzijds zorgt ze ervoor dat de gebruikers op een toegankelijke manier met dependency-gegevens en AI-analyses kunnen werken. Anderzijds vormt ze de technische verbindingslaag die de verschillende backend- en AI-componenten samenbrengt tot een samenhangend systeem. Hierdoor wordt DepGuard AI niet louter een verzameling losse analysecomponenten, maar een geïntegreerd dashboard dat projectgegevens, update-informatie en AI-ondersteuning samen aanbiedt in een workflow.

6

Resultaten en evaluatie

Dit hoofdstuk bespreekt de resultaten van de uitgevoerde evaluatie van de drie AI-varianten binnen DepGuard AI. De analyse vertrekt vanuit de benchmarkuitvoer van de proof-of-concept en vergelijkt de oplossingen op het vlak van specificiteit, volledigheid, actiegerichtheid en verwerkingstijd. Op basis daarvan wordt nagegaan in welke mate de verschillende benaderingen bruikbare ondersteuning bieden voor dependencybeheer in de context van Springbok.

6.1. Evaluatiekader

De evaluatie werd uitgevoerd op basis van een benchmarkdataset met twaalf uitgewerkte testgevallen, verspreid over drie representatieve updatecategorieën: een minor update van React, een minor update van Next.js, een major update van TypeScript en een patch update van Zod. Elke case werd driemaal uitgevoerd per agent, waardoor per oplossing een vergelijkbare set resultaten beschikbaar is. Voor elke analyse werd een score toegekend op vier criteria: specificiteit, volledigheid, actiegerichtheid en algemene kwaliteit.

Naast de inhoudelijke beoordeling werd ook de verwerkingstijd gemeten. Daarmee kan niet enkel worden gekeken naar de kwaliteit van de analyse, maar ook naar de praktische haalbaarheid van de oplossing in een interactieve dashboard-context. De benchmark bevat bovendien beoordelingscommentaar per run, waardoor naast de numerieke score ook kwalitatieve verschillen tussen de agent zichtbaar worden.

Tabel 6.1 geeft een overzicht van de gebruikte testcasegroepen, terwijl Tabel 6.2 de geaggregeerde resultaten per AI-variant samenvat.

Tabel 6.1: Overzicht van de gebruikte testcasegroepen

Testcase	Pakket	Update-type
React minor	react	minor
Next.js minor	next	minor
TypeScript major	typescript	major
Zod patch	zod	patch

Tabel 6.2: Samenvatting van de benchmarkresultaten per AI-variant

Variant	Algemene score	Specificiteit	Volledigheid	Verwerkingstijd
OpenAI	8,71	8,33	9,60	5,18 s
Mastra	8,47	8,07	9,33	11,66 s
n8n	8,25	7,93	8,60	7,02 s

6.2. Globale prestaties

Op basis van de geaggregeerde resultaten behaalt OpenAI de hoogste gemiddelde totaalscore met 8,71 op 10, gevolgd door Mastra met 8,47 en n8n met 8,25. OpenAI scoort bovendien het hoogst op volledigheid en actiegerichtheid, met respectievelijk 9,6 en 8,6 als gemiddelde, terwijl Mastra en n8n iets lager uitkomen op die dimensies. Dat wijst erop dat de OpenAI-variant de meest consistente balans biedt tussen inhoudelijke dekking en bruikbare samenvatting.

Wat verwerkingstijd betreft is OpenAI eveneens het snelst, met een gemiddelde latency van ongeveer 5,18 seconden. n8n volgt met ongeveer 7,02 seconden, terwijl Mastra duidelijk trager is met gemiddeld 11,66 seconden per analyse. Die verschillen zijn relevant in een dashboardcontext, omdat de resultaten niet alleen correct moeten zijn, maar ook snel genoeg beschikbaar moeten worden gemaakt om de gebruikerservaring vlot te houden.

6.3. Per agent

De OpenAI-variant levert in de meeste gevallen de meest compacte maar tegelijk volledige samenvattingen op. In de benchmark worden vaak alle verwachte onderdelen opgenomen, namelijk nieuwe functies, breaking changes, migratiestappen en een expliciete veiligheidsinschatting. De beoordelingen wijzen erop dat deze aanpak vooral sterk is wanneer ontwikkelaars snel een onderbouwde beslissing moeten nemen zonder zelf nog door de volledige changelog te moeten gaan.

Mastra scoort het hoogste wanneer inhoudelijke volledigheid centraal staat. De analyses bevatten doorgaans veel detail over nieuwe functies, breaking changes en migratiestappen, en de beoordelingscommentaren tonen aan dat deze variant vaak las specifiek en bruikbaar wordt ervaren. Tegelijk is Mastra ook de traagste

aanpak, wat suggereert dat de extra volledigheid gepaard gaat met een merkbare kost op vlak van doorlooptijd. Voor een interactief dashboard is dat een belangrijk aandachtspunt.

De n8n-variant presteert gemiddeld degelijk, maar toont meer variantie in volledigheid. In meerdere testgevallen ontbreken migratiestappen of blijft de inhoud van bepaalde secties beperkter, ook al blijft de eindconclusie vaak bruikbaar voor een snelle beslissing. Die combinatie maakt n8n functioneel inzetbaar, maar inhoudelijk minder sterk dan de andere twee oplossingen wanneer de analyse expliciet ondersteuning moet bieden bij het inschatten van impact en vervolgstappen.

6.4. Vergelijking per testcase

Bij de React minor update valt op dat OpenAI de meest evenwichtige analyse oplevert. De output bevat duidelijk duiding van de wijziging, een correcte veiligheidsinschatting en voldoende context om de update zonder bijkomende lezing te beoordelen. Mastra levert hier meer detail over deprecated APIs en mogelijke aandachtspunten, terwijl n8n soms minder volledig is in de migratieinformatie.

Bij de Next.js minor update blijft hetzelfde patroon zichtbaar. OpenAI en Mastra leveren beide bruikbare en inhoudelijk sterke analyses, maar OpenAI blijft sneller en consistent in structuur. n8n haalt ook hier bruikbare resultaten, maar de inhoudelijke volledigheid varieert sterker per run, wat de voorspelbaarheid van de output vermindert.

De TypeScript major update vormt de moeilijkste testcase, omdat hier de impact van breaking changes groter is. De evaluatie toont dat alle drie de aanpakken de ernst van de update herkennen, maar OpenAI en Mastra geven meer context over de gevolgen voor de codebase en de nood aan aanpassing. n8n blijft bruikbaar, maar de feedback wijst vaker op een beperktere concreetheid in de aanbevelingen. Bij de Zod patch update presteren de drie varianten het best, zoals ook verwacht mag worden bij een kleine versieaanpassing. OpenAI en Mastra geven duidelijke en veilige aanbevelingen, terwijl n8n meestal een correcte maar minder rijke samenvatting oplevert. Dit bevestigt dat de oplossing vooral sterk is wanneer releaselinformatie beperkt is en de update relatief weinig risico inhoudt.

6.5. Interpretatie van de resultaten

De benchmarkresultaten tonen aan dat het mogelijk is om dependency-updates automatisch te analyseren en te vertalen naar begrijpelijke samenvatting die bruikbaar zijn voor ontwikkelaars. Tegelijk blijkt dat de kwaliteit van de output sterk afhangt van de gekozen implementatievorm. De resultaten ondersteunen daarmee de hypothese dat een AI-gestuurde laag waardevol kan zijn binnen dependency-beheer, maar dat de praktische meerwaarde niet alleen wordt bepaald door het taalmodel zelf, maar ook door de orkestratie, bronselectie en fallbacklogica.

De evaluatie bevestigt ook dat geen enkele variant in alle opzichten dominant is. OpenAI is het snelst en scoort het hoogste op algemene kwaliteit, Mastra is het meest gedetailleerd maar trager, en n8n is functioneel bruikbaar maar minder stabiel op vlak van volledigheid. Voor de context van Springbok betekent dit dat de keuze voor een implementatie niet uitsluitend op outputkwaliteit kan worden gebaseerd, maar ook op onderhoudbaarheid, integratiemogelijkheden en gewenste reactietijd.

6.6. Antwoord op de onderzoeksvraag

De resultaten tonen aan dat een AI-gedreven agent effectief kan worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren. In de praktijk gebeurt dit door versie-informatie, release notes en changelogs te verzamelen, te structureren en om te zetten naar een samenvatting die het kernteam toelaat sneller te beslissen. De benchmark laat zien dat deze aanpak bruikbare output oplevert voor zowel kleine als grote updates, met een duidelijke meerwaarde voor het inschatten van impact en prioriteit.

Daarmee wordt de onderzoeksvraag bevestigend beantwoord: een AI-gestuurde analyselaag kan het manuele werk rond dependency-updates beperken, op voorwaarde dat de informatiebronselectie, de outputstructuur en de integratie binnen het dashboard voldoende doordacht zijn. De beste resultaten worden behaald wanneer de agent niet enkel samenvat, maar ook context bewaart en een heldere veiligheidsinschatting formuleert.

7

Conclusie

Deze bachelorproef onderzocht hoe een AI-gedreven agent kan worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dependency-updates efficiënter en met minder handmatig werk kan verwerken. Op basis van de literatuurstudie, requirementsanalyse, implementatie en evaluatie kan worden geconcludeerd dat een AI-ondersteunde aanpak inderdaad een relevante meerwaarde biedt voor dependencybeheer binnen Springbok.

De eerste deelvraag betrof de belangrijkste uitdagingen en risico's van het huidige dependency management. Daaruit bleek dat de developers geconfronteerd worden met een combinatie van verouderde dependencies, technische schuld en een toenemende hoeveelheid informatie die manueel moet worden geïnterpreteerd. Uit de stand van zaken kwam naar voren dat het probleem niet alleen in de aanwezigheid van updates ligt, maar vooral in de kloof tussen detectie en besluitvorming. Updates worden vaak wel gesignaleerd, maar de context om te bepalen hoe belangrijk of dringend ze zijn, moet nog manueel worden verzameld en geïnterpreteerd.

De tweede deelvraag ging over bestaande tools en hun beperkingen. De literatuur en de uitgezekerde implementatie tonen aan dat klassieke dependencybots nuttig zijn voor detectie en notificatie, maar dat ze de inhoudelijke beoordeling van changelogs, release notes en mogelijke impact grotendeels overlaten aan de ontwikkelaar. Daardoor wordt de taak van het kernteam niet weggehaald, maar verschuift ze grotendeels naar interpretatie en prioritering. In dat opzicht vullen AI-agents een duidelijke lacune op, omdat zij die informatie kunnen samenvatten en structureren.

De derde deelvraag handelde over technische schuld. De resultaten bevestigen dat uitgestelde dependency-updates de onderhoudslast verder doen toenemen, omdat elke bijkomende update een grotere context en meer afhankelijkheden mee-

brengt. Dat maakt het steeds moeilijker om updates op een efficiënte manier door te voeren. De bijdrage van DepGuard AI ligt precies in het verminderen van die interpretatielast, zodat updates sneller kunnen worden beoordeeld voordat de achterstand verder oploopt.

De vierde deelvraag focuste op de inzet van AI-agents en workflowautomatisering. De implementatie toont dat een agentische aanpak geschikt is om informatie uit changelogs, release notes en andere bronnen samen te brengen tot een bruikbare samenvatting. De combinatie van toolgebruik, bronselectie en gestructureerde output maakt het mogelijk om uiteenlopende updategegevens in een doorlopende analyseketen te verwerken. Daardoor ontstaat een systeem dat niet enkel detecteert, maar ook duidt.

De vijfde deelvraag betrof de functionele en niet-functionele vereisten van een dergelijk platform. Uit de requirementsanalyse bleek dat het systeem onder meer moet kunnen integreren met bestaande ontwikkelomgevingen, overzicht moet bieden per project en dependency, en voldoende traceerbaar, uitbreidbaar en onderhoudbaar moet zijn. De uitgewerkte architectuur en implementatie tonen aan dat deze vereisten richtinggevend waren voor de opbouw van het proof-of-concept. Vooral de scheiding tussen frontend, backend, databank en AI-laag bleek belangrijk om het systeem beheersbaar te houden.

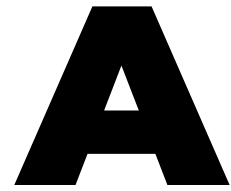
De zesde deelvraag onderzocht in welke mate de geselecteerde AI-agent- en workflowplatformen voldoen aan deze vereisten. De vergelijking liet zien dat geen enkele oplossing in alle opzichten superieur is. OpenAI scoorde het sterkst op snelheid en algemene kwaliteit, Mastra leverde de meest volledige en inhoudelijk rijke output, en n8n bood een functionele maar minder consistente variant. Deze resultaten tonen aan dat de uiteindelijke keuze afhankelijk is van het gewicht dat wordt toegekend aan snelheid, detailniveau en onderhoudbaarheid binnen de specifieke context van Springbok.

De zevende deelvraag peilde naar de effectieve prestaties van de agents. De benchmarkresultaten tonen dat de drie varianten in staat zijn om dependency-updates zinvol te analyseren, maar dat ze onderling verschillen in volledigheid, specificiteit en reactietijd. Vooral OpenAI leverde de meest evenwichtige combinatie van snelheid en bruikbaarheid, terwijl Mastra meer detail gaf tegen een hogere verwerkingstijd. Dat bevestigt dat AI-gestuurde analyse waardevol is, maar dat de optimale implementatie afhangt van de gewenste balans tussen snelheid en inhoudelijke rijkdom.

Algemeen kan worden gesteld dat deze bachelorproef een werkbare basis heeft aangetoond voor AI-ondersteund dependencybeheer. De meerwaarde van DepGuard AI ligt niet in het volledig automatiseren van beslissingen, maar in het verminderen van de handmatige analysetijd en het beter structureren van update-informatie voor het kernteam. Daarmee wordt een praktische bijdrage geleverd aan het beperken van technische schuld en het verhogen van de efficiëntie binnen het de-

pendencybeheerproces.

Tegelijk zijn er ook beperkingen aan deze studie. De evaluatie is uitgevoerd op basis van een beperkt aantal representatieve testgevallen en binnen de tijds- en scopegrenzen van een bachelorproef. Daardoor kunnen de resultaten niet zonder meer worden veralgemeend naar alle soorten project of dependency-ecosystemen. Verder onderzoek kan zich richten op bredere datasets, meer verfijnde evaluatiecriteria en een diepere integratie met productieomgevingen. Ook een vergelijking met bijkomende agent- of workflowplatformen kan toekomstige ontwerpkeuzes verder onderbouwen.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden. (Pashchenko et al., 2018) In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

A.1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld. (Behutiye et al., 2024; Pashchenko et al., 2018) Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt. Pashchenko et al. (Pashchenko et al., 2018) stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als

technische schuld (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn.(Behutiye et al., 2024; Ruiz et al., 2024) Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit.(Besker, 2020) Om deze last te verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn,(Erlenhov et al., 2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd.(Erlenhov et al., 2022) Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van het bedrijf. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm,³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek *from scratch* wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol. De webapplicatie leeft dus als een losstaand platform naast GitHub, maar gebruikt GitHub uitsluitend als bron voor project- en dependency-informatie en, eventueel, als kanaal om later pull requests of issues te creëren. Op die manier kan het updateproces slimmer, sneller en inzichtelijker worden gemaakt.

A.1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform-

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt.<https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt.<https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem.<https://www.npmjs.com>

en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?
2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van dit bedrijf?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen het bedrijf?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van het bedrijf?

A.1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde, from-scratch ontworpen AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van het bedrijf voor geautomatiseerd dependencybeheer. Eerder onderzoek toont aan dat AI-technieken succesvol kunnen worden ingezet binnen onderhoudstaken zoals changelog-analyse en code review, wat aantoont dat deze toepassing haalbaar en relevant is. (Behutiye et al., 2024; Joshi, 2025; Kim et al., 2024; Zhu et al., 2025)

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registers (zoals npm);
- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van het bedrijf. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van het bedrijf;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen het bedrijf.

A.2. Literatuurstudie

A.2.1. Dependency management en kwetsbare dependencies

Moderne softwareprojecten steunen sterk op open-source dependencies, waardoor kwetsbaarheden in externe bibliotheken rechtstreeks een impact kunnen hebben op de veiligheid en betrouwbaarheid van toepassingen. (Pashchenko et al., 2018) Pashchenko et al. tonen aan dat een groot deel van de aangetroffen kwetsbare dependencies in de praktijk relatief eenvoudig verholpen kan worden door te upgraden naar een veiligere versie, maar dat ontwikkelteams deze updates vaak niet consequent doorvoeren. (Pashchenko et al., 2018)

In hun proefondervindelijk onderzoek onderscheiden Pashchenko et al. tussen daadwerkelijk gedeployde kwetsbare dependencies en libraries die enkel tijdens ontwikkeling of testing gebruikt worden, en stellen zij vast dat het merendeel van de reële kwetsbare dependencies door de eigen ontwikkelaars of directe onderhouders kan worden opgelost. (Pashchenko et al., 2018) Dit onderstreept het belang van systematisch dependencybeheer en duidelijke inzichten in welke dependencies effectief risico vormen in productie omgevingen. (Pashchenko et al., 2018)

A.2.2. Technische schuld en onderhoudslast

Het uitstellen van dependency-updates draagt bij aan de opbouw van technische schuld, waardoor onderhoud en toekomstige wijzigingen steeds meer inspanning vereisen.(Behutiye et al., 2024; Ruiz et al., 2024) Behutiye et al. analyseren de rol van technische schuld in agile omgevingen en beschrijven hoe keuzes die op korte termijn tijdswinst opleveren, op langere termijn leiden tot hogere kosten en complexere wijzigingen.(Behutiye et al., 2024)

Ruiz et al. bespreken in hun tertiaire studie dat technische-schuldmanagement de voorbije jaren een groeiend onderzoeksdomein is geworden, waarbij organisaties worstelen met het in kaart brengen, prioriteren en terugdringen van opgebouwde schuld.(Ruiz et al., 2024) Empirisch werk van Chalmers University of Technology bevestigt dat technische schuld niet louter een theoretisch concept is, maar in industriële context concreet voelbaar is in de vorm van verminderde productiviteit, hogere foutkans en vertragingen in softwarelevering.(Besker, 2020)

A.2.3. Dependencybots en hun beperkingen

Om de handmatige last rond dependencybeheer te verlagen, zetten veel projecten dependency management bots in, zoals Dependabot en Renovate.(Erlenhov et al., 2022) Erlenhov et al. voeren een kwantitatieve en kwalitatieve studie uit naar dependencybots in open-source systemen en stellen vast dat slechts een beperkt aantal van de geanalyseerde geautomatiseerde tools voldoet aan de criteria van volwaardige “Devbots”, die autonoom bijdragen aan de codebasis.(Erlenhov et al., 2022)

Hun resultaten tonen dat projecten vaak experimenteren met meerdere bots en regelmatig wisselen, onder meer door problemen met ruis (te veel pull requests), beperkte configureerbaarheid en moeilijkheden om de impact van voorgestelde updates goed te begrijpen.(Erlenhov et al., 2022) Hoewel dependencybots het detecteren en aanmaken van update-pull-requests automatiseren, blijft de inhoudelijke beoordeling van changelogs, compatibiliteit en risico's grotendeels bij ontwikkelaars liggen, wat aansluit bij de probleemstelling van deze bachelorproef.(Erlenhov et al., 2022; Pashchenko et al., 2018)

A.2.4. AI, agents en workflow automatisering

Recente literatuur verkent hoe AI-agents en multi-agentframeworks kunnen worden ingezet om complexe taken in softwareontwikkeling en data-intensieve workflows te coördineren.(Joshi, 2025; Kim et al., 2024; Zhu et al., 2025) Joshi biedt een overzicht van autonome systemen en collaboratieve AI-agentframeworks en benadrukt dat zulke agents vooral nuttig zijn in scenario's met veel herhalende beslissingen op basis van tekstuele en gestructureerde input, zoals logs, notificaties en rapporten.(Joshi, 2025)

Kim et al. introduceren met Bel Esprit een multi-agentframework voor het opbou-

wen van AI-model-pijplijnen, waarbij verschillende gespecialiseerde agents samenwerken om complexere taken modulair af te handelen.(Kim et al., 2024) Zhu et al. bestuderen in OAgents empirisch welke ontwerpkeuzes leiden tot effectieve agents, en bespreken onder meer het belang van duidelijke taakafbakening, robuuste toolintegratie en feedbackloops voor betrouwbare beslissingsondersteuning.(Zhu et al., 2025) Deze inzichten zijn relevant voor het ontwerp van een AI-agent die dependency-updates interpreteert, samenvat en beslissingsvoorstellen formuleert voor een onderhoudsteam.

Naast generieke agentframeworks beschrijft Wali et al. hoe workflow-automatisering met n8n⁴ operationele efficiëntie kan verhogen door repetitieve, gegevensgedreven processen te coördineren over verschillende systemen heen.(Wali et al., 2025) Hoewel hun casus zich richt op een financiële context, illustreert de studie dat low-code workflowtools geschikt zijn om integraties met externe APIs, databronnen en notificatiekanalen te realiseren, wat ook essentieel is voor een geautomatiseerde dependency-updatepipeline.(Wali et al., 2025)

A.2.5. Onderzoeksniche: AI-ondersteund dependencybeheer

Samengenomen schetst de literatuur een duidelijk beeld: kwetsbare of verouderde dependencies vormen een reëel risico, maar kunnen vaak verholpen worden door updates die vandaag onvoldoende systematisch worden uitgevoerd.(Behutiye et al., 2024; Pashchenko et al., 2018) Tegelijk tonen studies naar technische schuld dat het structureel uitstellen van onderhoud, waaronder dependency updates, leidt tot een groeiende onderhoudslast en achterstand.(Behutiye et al., 2024; Besker, 2020; Ruiz et al., 2024)

Onderzoek naar dependencybots maakt duidelijk dat huidige oplossingen weliswaar updates detecteren en pull requests genereren, maar ontwikkelaars nog steeds belasten met het interpreteren van changelogs en het beoordelen van risico's en prioriteiten.(Erlenhov et al., 2022) De recente vooruitgang in AI-agents en workflow-automatisering suggereert dat een volgende stap mogelijk is: een systeem waarin een AI-agent niet alleen updates signaleert, maar ook de inhoud van release notes en changelogs analyseert, samenvat en contextafhankelijk advies geeft aan een klein kernteam dat verantwoordelijk is voor dependencybeheer.(Joshi, 2025; Kim et al., 2024; Wali et al., 2025; Zhu et al., 2025)

Deze bachelorproef positioneert zich precies in deze niche door een AI-gestuurde laag bovenop bestaande dependencybots te onderzoeken en te prototypen, met de expliciete focus op het verlagen van de manuele beoordelingslast en het beheersbaar houden van technische schuld rond dependencies in een bedrijfscontext.

⁴n8n is een open-source tool om workflows en API's visueel te automatiseren.<https://n8n.io>

A.3. Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

A.3.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van het bedrijf geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

A.3.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kriticiiteit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en

terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

A.3.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een aparte webapplicatie (dashboard) waarin per project de gebruikte dependencies worden bijgehouden en verrijkt met informatie uit publieke registries zoals npm. De applicatie werkt vanaf het begin met één of meerdere reële GitHub-repositories van het bedrijf, zodat de proof-of-concept onmiddellijk getest wordt in een realistische context. Dependency-informatie wordt automatisch uit deze repositories gehaald (bijvoorbeeld via repositoryconfiguratiebestanden) en opgeslagen in een centrale databank.

Parallel wordt de AI-agentlaag geïmplementeerd in de backend. In deze fase worden drie concrete AI-agentoplossingen opgezet en geïntegreerd in hetzelfde dashboard: een agent op basis van OpenAI Agent Builder, een workflow met n8n en een agentconfiguratie met Flowise. Elk van deze varianten ontvangt changelogs en release-note-informatie, genereert beknopte samenvattingen en beoordeelt de aard en vermoedelijke impact van updates (bijvoorbeeld beveiligingsfix, bugfix, nieuwe functionaliteit, potentieel brekende wijziging). De frontend visualiseert de status van dependencies, aanbevolen acties en de door de verschillende AI-varianten gegenereerde toelichting. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar GitHub en één dependency-registries;
- drie geïmplementeerde AI-agentvarianten (OpenAI Agent Builder, n8n, Flowise) die changelogs en release notes verwerken en samenvatten;
- technische documentatie over de implementatie (architectuur, API's, datamodellen en integratiepunten voor de agents).

A.3.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requi-

rements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Flowise onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

A.3.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd met de co-promotor binnen het bedrijf, aan de hand van een combinatie van demonstraties, gericht feedbackgesprekken en, waar mogelijk, beperkte praktijktesten realistische projecten. De evaluatie focust op bruikbaarheid van het dashboard, de gemeten vermindering van manuele analysetijd (bijvoorbeeld aantal geanalyseerde updates per uur vóór en na gebruik), de door de co-promotor toegekende beoordelingsscores voor duidelijkheid van de AI-samenvattingen en de mate waarin het systeem helpt bij de prioritering van updates.

De bevindingen uit deze evaluatie worden gecombineerd met de kwantitatieve resultaten van fase 4 om de onderzoeksvragen te beantwoorden en de onderzoekshypothesen te toetsen. Deliverables van fase 5 zijn:

- een synthese van de evaluatie met de co-promotor (inclusief gemeten tijdswinst, kwaliteitsbeoordelingen en verbetervoorstellen);
- de uitgewerkte bachelorproef met een geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling en mogelijke integratie van het systeem in de bredere ontwikkelprocessen van het bedrijf.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur A.1.

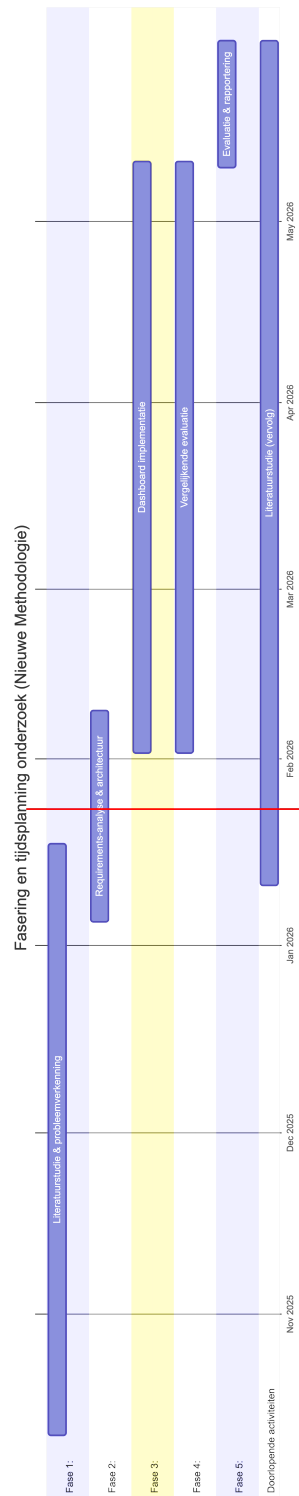
A.4. Verwacht resultaat en conclusie

Op basis van de probleemstelling en onderzoeksdoelstelling wordt verwacht dat het proof-of-concept in staat zal zijn om dependency-updates automatisch te detecteren, te analyseren en in een compacte, begrijpelijke vorm te presenteren. De AI-agent zal changelogs en release notes samenvatten, het type wijziging (bijvoorbeeld beveiligingsfix, bugfix of nieuwe functionaliteit) aanduiden en een indicatie geven van de vermoedelijke impact op het project.

Verder wordt verwacht dat de werklast daalt doordat een deel van de beslissingen rond relatief veilige, backward compatible updates (zoals patch- of beperkte minor-releases) eenvoudiger of gedeeltelijk geautomatiseerd kan worden, terwijl potentieel risicovolle major-updates expliciet voor manuele review gemarkeerd blijven. Indien er voldoende meetdata beschikbaar komt (bijvoorbeeld logbestanden of tijdsregistraties), kan een eerste, kwantitatieve inschatting gemaakt worden van de tijdswinst in vergelijking met de huidige, hoofdzakelijk manuele aanpak.

De doelgroep, haalt hieruit meerwaarde doordat zij sneller prioriteiten kunnen bepalen, minder tijd verliezen aan het doorpluizen van changelogs en release notes en een beter overzicht krijgen van de actuele staat van dependencies en de bijhorende technische schuld. Daarnaast ontstaat een herhaalbaar proces dat later kan opgeschaald worden naar andere teams of projecten binnen de organisatie.

Verwacht wordt dat de evaluatie van de gekozen AI-agent- en/of workflowaanpak zal tonen dat geen enkele oplossing op alle criteria de beste is, maar dat duidelijke verschillen zichtbaar worden op het vlak van integratiemogelijkheden, gebruiksgemak, onderhoudbaarheid en performantie. Op basis van deze inzichten kan een onderbouwde aanbeveling worden geformuleerd voor een architectuur en toolkeuze die het best aansluit bij de context van het bedrijf, en kan geconcludeerd worden dat een AI-gestuurde laag een zinvolle stap is in het beperken van technische schuld rond dependencies. Indien de uiteindelijke resultaten afwijken van deze verwachtingen, biedt dit aanleiding om te onderzoeken welke aannames niet bleken te kloppen en welke randvoorwaarden essentieel zijn voor succesvol AI-ondersteund dependencybeheer.



Figuur A.1: Fasering en tijdsplanning van het onderzoek

Bibliografie

- AI, M. (2025). *Observability Overview* [Documentatie over tracing en observability voor Mastra-agents en workflows]. Geraadpleegd op 2 maart 2026, van <https://mastra.ai/docs/observability/overview>
- AI, M. (2026). *Mastra: The TypeScript AI Agent Framework* [Open-source framework voor het bouwen van AI-agents en workflows in TypeScript]. Geraadpleegd op 2 maart 2026, van <https://github.com/mastra-ai/mastra>
- Behutiye, W. N., Rodriguez, P., Oivo, M., & Tosun, A. (2024). Analyzing the Concept of Technical Debt in the Context of Agile Software Development: A Systematic Literature Review. *arXiv*. <https://arxiv.org/abs/2401.14882>
- Besker, T. (2020). *Technical Debt: An Empirical Investigation of Its Harmfulness and Management Strategies in Industry* [PhD thesis]. Chalmers University of Technology. <https://research.chalmers.se/en/publication/518318>
- Erlenhov, L., De Oliveira Neto, F. G., & Leitner, P. (2022). Dependency Management Bots in Open-Source Systems—Prevalence and Adoption. *PeerJ Computer Science*, 8, e849. <https://doi.org/10.7717/peerj-cs.849>
- Hatchworks. (2025). *n8n Guide 2026: Features & Workflow Automation Deep Dive* [Diepgaande gids over n8n met nadruk op AI-workflows, triggers en integraties]. Geraadpleegd op 9 maart 2026, van <https://hatchworks.com/blog/ai-agents/n8n-guide/>
- Joshi, S. (2025). Review of Autonomous Systems and Collaborative AI Agent Frameworks. *International Journal of Science and Research Archive*, 14(02), 961–972. <https://doi.org/10.30574/ijrsra.2025.14.2.0439>
- Kim, Y., Abdelaziz, A. E., Castro Ferreira, T., Al-Badrashiny, M., & Sawaf, H. (2024). Bel Esprit: Multi-Agent Framework for Building AI Model Pipelines. *arXiv*. <https://doi.org/10.48550/arXiv.2412.14684>
- n8n GmbH. (2025). *n8n Documentation: About n8n* [Officiële documentatie over n8n als workflow-automatiseringsplatform]. Geraadpleegd op 9 maart 2026, van <https://docs.n8n.io>
- OpenAI. (2026a). Agents SDK | OpenAI API [Bezocht op 2026-04-20]. <https://developers.openai.com/api/docs/guides/agents>
- OpenAI. (2026b). Function calling | OpenAI API [Bezocht op 2026-04-20]. <https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI. (2026c). Using tools | OpenAI API [Bezocht op 2026-04-20]. <https://developers.openai.com/api/docs/guides/tools>

- Pashchenko, I., Plate, H., Ponta, S. E., Sabetta, A., & Massacci, F. (2018). Vulnerable Open Source Dependencies: Counting Those That Matter. *Proceedings of the 12th International Symposium on Empirical Software Engineering and Measurement (ESEM)*.
- Proser, Z. (2025). *n8n: The Workflow Automation Tool for the AI Age* [Artikel dat de architectuur, event-gedreven werking en AI-toepassingen van n8n beschrijft]. Geraadpleegd op 9 maart 2026, van <https://workos.com/blog/n8n-the-workflow-automation-tool-for-the-ai-age>
- Ruiz, J. A., Aguilar, R. A., Garcilazo, J. F., & Aguilera, A. A. (2024). A Tertiary Study on Technical Debt Management over the last lustrum. *International Journal of Combinatorial Optimization Problems and Informatics*, 15(5), 181–192. <https://doi.org/10.61467/2007.1558.2024.v15i5.577>
- Tavily. (2026a). *What is the Tavily Search API?* [Help center article]. Geraadpleegd op 27 april 2026, van <https://help.tavily.com/articles/4840311948-tavily-search-api>
- Tavily. (2026b, februari). *Best Practices for Search* [Tavily documentation]. Geraadpleegd op 27 april 2026, van <https://docs.tavily.com/documentation/best-practices/best-practices-search>
- Tavily. (2026c, februari). *Introduction* [Tavily API reference]. Geraadpleegd op 27 april 2026, van <https://docs.tavily.com/documentation/api-reference/introduction>
- Tavily. (2026d, april). *Web Search Essentials* [Tavily documentation]. Geraadpleegd op 27 april 2026, van <https://docs.tavily.com/examples/quick-tutorials/search-api>
- Tavily AI. (2025, december). *Introduction* [Alternative Tavily documentation mirror]. Geraadpleegd op 27 april 2026, van <https://tavilyai.mintlify.app/documentation/api-reference/introduction>
- The AI Builders. (2025). *Mastra: Build Powerful AI Agents with Typescript* [Tutorial met voorbeeldagents, tools en workflows in Mastra]. Geraadpleegd op 2 maart 2026, van https://tutorial.theaibuilders.dev/tutorials/Frameworks/mastra_tutorial
- Wali, M., Nasir, N., & Iqbal, T. (2025). Implementing Workflow Automation with n8n to Enhance Operational Efficiency and Performance in the Sharia Cooperative of Bank Indonesia, Aceh Province. *Journal Digital Technology Trend*, 4(1), 36–47. <https://doi.org/10.56347/jdtt.v4i1.341>
- Zhu, H., Qin, T., Zhu, K., Huang, H., Guan, Y., Xia, J., Yao, Y., Li, H., Wang, N., Liu, P., Peng, T., Gui, X., Li, X., Liu, Y., Jiang, Y. E., Wang, J., Zhang, C., Tang, X., Zhang, G., ... Zhou, W. (2025). OAgents: An Empirical Study of Building Effective Agents. <https://arxiv.org/abs/2506.15741>